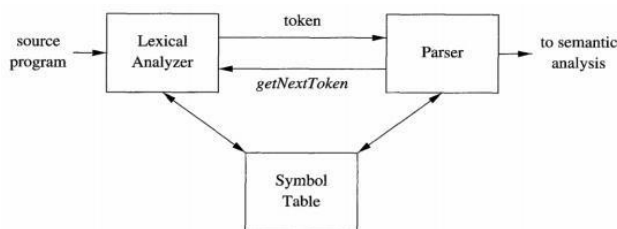


Chapter 2: Lexical Analysis

The Role of Lexical Analyzer

- Read the input characters of the source program, group them into lexemes, and produces a sequence of tokens
- Add lexemes into the symbol table when necessary
- Strip out comments and whitespace (blank, newline, tab etc.)
- Associate error messages with a line number (tracking newlines)



作为编译器的第一阶段，词法分析器的主要任务是读取源程序的输入字符，将它们组成为词素，生成并输出一个词法单元序列，每个词法单元对应一个词素。这个词法单元序列被输出到语法分析器进行语法分析。这些相互作用如图 3.1 所示。通常，交互是通过让语法分析器调用词法分析器来实现的。 `getNextToken` 命令所指示的调用使词法分析器从其输入中读取字符，直到它可以识别下一个词素为止。词法分析器根据这个词素生成下一个词法单元并将其返回给语法分析器。

- 词法分析器与符号表交互也很常见。当词法分析器发现构成标识符的词素时，它需要将该词素输入到符号表中。在某些情况下，词法分析器可以从符号表中读取有关标识符种类的信息，以帮助它确定必须传递哪个词法单元。

由于词法分析器是编译器中读取源文本的部分，因此除了词素识别之外，它还可以执行某些其他任务。

- 其中一项任务是去除注释和空格（空白、换行符、制表符以及可能用于分隔输入中词法单元的其他字符）。

- 另一项任务是将编译器生成的错误消息与源程序的位置联系起来。例如，词法分析器可以跟踪所遇到的换行符的数量，因此它可以给每个错误消息赋予一个行号。在某些编译器中，词法分析器会制作源程序的副本，并将错误消息插入到适当的位置。如果源程序使用宏预处理器，则宏的扩展也可以由词法分析器完成。

Why Separated Lexical Analysis?

- **Simplicity of compiler design.** The lexical analyzer can perform simple tasks such as dealing with comments and whitespace
- **Improved compiler efficiency.** Lexical analysis is much simpler comparing to syntax analysis
- **Higher portability of compilers.** Only lexical analyzer needs to deal with input-device-specific peculiarities (e.g., line separator)

出于多种原因，编译器的分析部分通常分为词法分析和解析（语法分析）阶段。

1.设计的简单性是最重要的考虑因素。词法分析和语法分析的分离通常使我们能够简化这些任务中的至少一项。例如，如果一个语法分析器必须将注释和空格作为语法单元进行处理，那么他就会比那些假设空白和注释已被词法分析器删除的语法分析器要复杂得多。如果我们正在设计一种新语言，分离词汇和语法问题有助于我们得到一个更加清晰的语言设计方案。

2.编译效率提高。把词法分析器独立出来使得我们能够使用专用于词法分析器、不进行语法分析的技术。此外，我们可以使用专门的用于读取输入字符的缓冲技术可以显著加快编译速度。

3.编译器可移植性增强。输入设备相关的特殊性可以被限制在词法分析器中。（如：行分隔）

Tokens, Patterns, and Lexemes

- A **token** is a pair $\langle \text{token name}, \text{attribute value} \rangle$
 - **Token name** is an abstract symbol representing a kind of **lexical unit**
 - **Attribute value** (optional) points to the symbol table
- A **pattern** is a description of the form that the lexemes of a token may take.
- A **lexeme** is a sequence of characters in the source program that matches the pattern for a token.
 - It is identified by the lexical analyzer as **an instance of the token**.

在讨论词法分析时，我们使用三个相关但不同的术语：

- **词法单元**是由词法单元名称和可选属性值组成的对。
 - 词法单元名称是表示某种词汇单元的抽象符号，例如，特定的关键字，或表示标识符的输入字符序列。词法单元名称是语法分析器处理的输入符号。后面的内容中，我们一般会用黑体字给出词法单元的名称。我们将使用词法单元的名字来引用一个词法单元
 - 属性值（可选）指向符号表
- **模式**描述了一个词法单元的词素可能具有的形式。当词法单元是一个关键字的情况下，它的模式就是组成这个关键字的字符序列。对于标识符和其他一些词法单元，模式是一个更复杂的结构，它可以和很多符号串匹配。
- **词素**是源程序中与某个词法单元模式匹配的一个字符序列，并被词法分析器识别为该词法单元的一个实例。

Examples

TOKEN	INFORMAL DESCRIPTION	SAMPLE LEXEMES
if	characters i, f	if
else	characters e, l, s, e	else
comparison	< or > or <= or >= or == or !=	<=, !=
id	letter followed by letters and digits	pi, score, D2
number	any numeric constant	3.14159, 0, 6.02e23
literal	anything but ", surrounded by "'s	"core dumped"

Consider the C statement: `printf("Total = %d\n", score);`

Lexeme	printf	score	"Total = %d\n"	(	...
Token	id	id	literal	left_parenthesis	...

例 3.1：图 3.2 给出了一些常见的词法单元、非正式描述的词法单元的模式，并给出了一些示例词素。下面说明上述概念在实践中是如何应用的。在 C 语句中

```
printf("Total = %d\n", score);
```

printf 和 score 都是和词法单元 id 的模式匹配的词素，而 “Total = %d\n” 则是一个与 literal 匹配的词素。

在许多编程语言中，以下类涵盖了大部分或全部词法单元：

1. 每个关键字有一个词法单元。关键字的模式就是该关键字本身。
2. 表示运算符的词法单元，它可以表示单个运算符，也如图 3.2 中的 comparison 那样，表示一类运算符。
3. 一个表示所有标识符的词法单元。
4. 一个或多个表示常量的词法单元，例如数字和字面值字符串。
5. 每个标点符号有一个词法单元，例如左括号、右括号、逗号和分号。

Attributes for Tokens

- When more than one lexeme match a pattern, the lexical analyzer must provide additional information, named *attribute values*, to the subsequent compiler phases
 - *Token names* influence parsing decisions
 - *Attribute values* influence semantic analysis, code generation etc.
- For example, an **id** token is often associated with: (1) its lexeme, (2) type, and (3) the location at which it is first found. Token attributes are stored in the *symbol table*.

`A = B * 2` $\longrightarrow$

- <**id**, pointer to symbol-table entry for A>
- <**assign_op**>
- <**id**, pointer to symbol-table entry for B> <**mult_op**>
- <**number**, integer value 2>

Fall 2024

BUPT Compilers

7

当多个词素可以匹配某个模式时，词法分析器必须向编译器的后续阶段提供有关被匹配词素的附加信息。例如，词法单元 `number` 的模式与 `0` 和 `1` 都能匹配，但是对于代码生成器来说，至关重要的是知道在源程序中找到了哪个词素。因此，在许多情况下，词法分析器不仅返回给语法分析器一个词法单元名字，还返回一个描述该词法单元所表示的词素的属性值；

- 词法单元名字将影响语法分析过程中的决策，
- 而这个属性值会影响语法分析后对这个词法单元的翻译。

我们将假设词法单元至多具有一个相关的属性值，当然该属性值可能是一个组合了多种信息的结构化数据。最重要的例子是词法单元 `id`，我们需要将大量信息与它关联。通常，和一个标识符有关的信息——例如，它的词素、类型以及它第一次出现的位置（在发出一个有关该标识符的错误消息时需要使用这个信息）——都保存在符号表中。因此，一个标识符的属性值是一个指向符号表中该标识符对应条目的指针。

示例 3.2: Fortran 语句 `A=B*2` 的词法单元名称和相关属性值在下面写为名字-属性对的序列。

请注意，在某些对中，特别是运算符、标点符号和关键字的对中，不需要属性值。在此示例中，词法单元 `number` 有一个整数属性值。实际上，编译器会保存一个代表该常量的字符串，并将指向该字符串的指针用作 `number` 的属性值。

Lexical Errors

- **The lexical analyzer is unable to proceed:** none of the patterns for tokens match any prefix of the remaining input
- Example: `int 3a = a * 3;`

如果没有其他组件的帮助，词法分析器很难发现源代码中的错误。比如，当词法分析器在 `c` 语言程序片段 `fi (a == f(x))` 中第一次遇到 `fi` 时，它无法指出 `fi` 究竟是关键字 `if` 的拼写错误还是一个未声明的函数标识符。由于 `fi` 是词法单元 `id` 的一个合法词素，因此词法分析器必须将词法单元 `id` 返回给语法分析器，并让编译器的某些其他阶段（在这种情况下是语法分析器）去处理这个由于字母换位而导致的错误。

然而，假设出现所有词法单元模式都无法与剩余输入的某个前缀匹配的情况，词法分析器就不能继续处理输入。最简单的恢复策略是“恐慌”恢复。我们从剩余的输入中不断删除字符，直到词法分析器能够，直到词法分析器可以在剩余输入的开头找到一个正确的词法单元为止。这种恢复技术可能会给语法分析器带来混乱，但是在交互式计算环境中，这可能已经足够了。

Specification of Tokens

- **Regular expression** (正则表达式, **regex** for short) is an important notation for specifying lexeme patterns
- Content of this part
 - Strings and Languages (串和语言)
 - Operations on Languages (语言上的运算)
 - Regular Expressions
 - Regular Definitions (正则定义)
 - Extensions of Regular Expressions

正则表达式是指定词素模式的重要符号。虽然它们不能表达所有可能的模式，但它们可以高效地描述在处理词法单元时要用到的模式类型。在本节中，我们将研究正则表达式的形式表示法本部分内容：

- 字符串和语言
- 语言上的运算
- 正则表达式
- 常规定义（正则定义）
- 正则表达式的扩展

Strings and Languages

- **Alphabet (字母表)**: any finite set of symbols
 - Examples of symbols: **letters**, **digits**, and **punctuations**
 - Examples of alphabets: **{1, 0}**, **ASCII**, **Unicode**
- A **string (串)** over an alphabet is a finite sequence of symbols drawn from the alphabet
 - The length of a string s , denoted $|s|$, is the number of occurrences of symbols in s (i.e., cardinality)
 - **Empty string (空串)**: the string of length 0, ϵ

字母表是一个有限的符号集合。

- 符号的典型示例是字母、数字和标点符号。
- 集合 $\{0, 1\}$ 是二进制字母表。ASCII 是字母表的一个重要例子；它被用于许多软件系统中。Unicode 包含了大约 100,000 个来自世界各地字母的字符，是字母表的另一个重要示例。

字母表上的字符串（串）是从字母表中抽取的符号的有限序列。字符串 s 的长度（用 $|s|$ 表示）是指有多少个

Strings and Languages Cont.

- String-related terms (using **university** for illustration)
 - **Prefix (前缀)** of string s : any string obtained by removing 0 or more symbols from the end of s (ϵ , **uni**, **univer**, **university**)
 - **Proper prefix (真前缀)**: a prefix that is not ϵ and not equal to s itself (**uni**, **univer**)
 - **Suffix (后缀)**: any string obtained by removing 0 or more symbols from the beginning of s (ϵ , **sity**, **versity**, **university**).
 - **Proper suffix (真后缀)**: a suffix that is not ϵ and not equal to s itself (**sity**, **versity**)

- 字符串相关术语（以 banana 为例进行说明）
 - 字符串 s 的前缀（前缀）：从 s 的末尾去掉 0 个或多个符号后得到的任何字符串
 - 适当的前缀（真前缀）：不是空且不为整个字符串
 - 后缀（Suffix）：从 s 开头去掉 0 个或多个符号后得到的字符串
 - 正后缀（真后缀）：不是空且不为字符串本身

Strings and Languages Cont.

- String-related terms (using **university** for illustration)
 - **Substring (子串)** of s: any string obtained by removing any prefix and any suffix from s (**ver**, **sity**, **uni**, **iv**, **ε**, **university**)
 - **Proper substring (真子串)**: a substring that is not ϵ and not equal to s itself (**ver**)
 - **Subsequence (子序列)**: any string formed by removing 0 or more not necessarily consecutive symbols from s (**uvrit**, **uiri**)



How many substrings does **napkin** have?

子串：从 s 中去掉任何前缀和后缀后得到的任何字符串, s 的所有前缀、后缀也都是它的子串。对任意的符号串 s , s 自身、 ϵ 都是 s 的前缀、后缀，也是 s 的子串。

真子串：不为空且不等于字符串

子序列：从 s 中删除 0 个或多个不一定连续的符号后形成的字符串

Let the length of the string be n , so looking at the example above, we understand that to find all possible numbers of substrings, we need to add substrings of length n , $(n-1)$, $(n-2)$, $(n-3)$, $(n-4)$, ..., 2, 1.

Total number of substrings = $n + (n - 1) + (n - 2) + (n - 3) + (n - 4) + \dots + 2 + 1$.

= $n * (n + 1) / 2 = 21$

So now we have a formula for evaluating the number of substrings where n is the length of a given string.

22 个 (考虑空串)

Strings and Languages Cont.

- String-related operations (串的运算)
 - **Concatenation (连接)**: the concatenation of two strings x and y , denoted xy , is the string formed by appending y to x
 - $x = \text{dog}, y = \text{house}, xy = \text{doghouse}$
 - **Exponentiation (幂/指数运算)**: $s^0 = \epsilon, s^1 = s, s^i = s^{i-1}s$
 - $x = \text{dog}, x^0 = \epsilon, x^1 = \text{dog}, x^3 = \text{dogdogdog}$

如果 x 和 y 是字符串, 则 x 和 y 的连接 (表示为 xy) 是把 y 附加到 x 后面而形成的字符串。例如, 如果 $x = \text{dog}$ 且 $y = \text{house}$, 则 $xy = \text{doghouse}$ 。空串是连接运算的单位元; 也就是说, 对于任何字符串 s 都有, $s = s = s$ 。

如果我们将两个串的连接视为这两个串的乘积, 我们可以如下定义字符串的“指数”运算如下: 定义 s^0 定义为 ϵ , 并且对于所有 $i > 0$, 将 s^i 为 $s^{i-1}s$ 。因为 $s = s$, 由此可知 $s^1 = s$, $s^2 = ss$, $s^3 = sss$, 依此类推。

Strings and Languages Cont.

- A **language** (语言) is any **countable set**¹ of strings over some fixed alphabets
 - The set containing only the empty string, that is $\{\epsilon\}$, is a language, denoted $\emptyset$
 - The set of all **grammatically correct English sentences**
 - The set of all **syntactically well-formed C programs**

¹In mathematics, a countable set is a set with the same cardinality (number of elements) as some subset of the set of natural numbers. A countable set is either a finite set or a countably infinite set.

- 语言 (语言) 是在某个固定字母表上的字符串的任何可数集合。

只包含空字符串的集合，即 $\{\epsilon\}$ ，即为空

所有语法正确的英语句子的集合

所有语法规范的 C 程序的集合

可数集 (Countable set)，是指每个元素都能与自然数集 N 的每个元素之间能建立一一对应的集合。如果将可数集的每个元素标上与它对应的那个自然数记号，那么可数集的元素就可以按自然数的顺序排成一个无穷序列 $a_1, a_2, a_3, \dots, a_n, \dots$ 。比如全体正偶数的集合是一个可数集，全体正奇数的集合也是可数集，它们与自然数集可以建立如下的一一对应。

Strings and Languages Cont.



- Operations on Languages (语言的运算)
 - 并，连接，Kleene闭包，正闭包

OPERATION	DEFINITION AND NOTATION
<i>Union of L and M</i>	$L \cup M = \{s \mid s \text{ is in } L \text{ or } s \text{ is in } M\}$
<i>Concatenation of L and M</i>	$LM = \{st \mid s \text{ is in } L \text{ and } t \text{ is in } M\}$
<i>Kleene closure of L</i>	$L^* = \bigcup_{i=0}^{\infty} L^i$
<i>Positive closure of L</i>	$L^+ = \bigcup_{i=1}^{\infty} L^i$

https://en.wikipedia.org/wiki/Stephen_Cole_Kleene

在词法分析中，最重要的语言上运算是并、连接和闭包。

并是常见的集合运算。语言的连接就是以各种可能的方式从第一种语言中任取一个字符串，再从第二语言任取一个串，然后将它们连接后得到的所有字符串的集合。

一个语言 L 的 (Kleene) 闭包，表示为 L^* ，就是将 L 连接 0 次或多次得到的字符串集合。注意 L^0 ，即“将 L 连接零次得到集合”被定义为 $\{g\}$ ，并且， L^i 被归纳地定义为 $L^{i-1}L$ 。

最后， L 的正闭包，表示为 L^+ ，与 Kleene 闭包基本相同，但没有项 L^0 。也就是说，除非属于 L ，否则不属于 L^+ 中。

Strings and Languages Cont.

- Operations on Languages (Example)

- $L = \{A, B, \dots, Z, a, b, \dots, z\}$
- $D = \{0, 1, \dots, 9\}$
- $L \cup D = \{A, B, \dots, Z, a, b, \dots, z, 0, 1, \dots, 9\}$
- LD : the set of 520 strings of length two, each consisting of one letter followed by one digit
- L^4 : the set of all 4-letter string
- L^* : the set of all strings of letters, including ϵ
- $L(L \cup D)^*$: ?
- D^+ : ?

例 3.3: 令 L 为字母的集合 $\{A, B, \dots, Z; a, b, \dots, z\}$ 并令 D 为数字集合 $\{0, 1, \dots, 9\}$ 。我们可以用两种本质上相同的方式来看待 L 和 D 。一种方式是 L 和 D 分别是大写字母和小写字母以及数字组成的字母表。第二种方式是 L 和 D 是语言, 其所有字符串的长度都恰好为 1。以下是可以使用图 3.6 中的运算符从语言 L 和 D 构造的一些新语言:

1. $L \cup D$ 是字母和数字的集——严格地说, 该语言有 62 个长度为 1 的字符串, 每个字符串要么是一个字母, 要么是一个数字。

2. LD 是 520 个长度为 2 的字符串的集合, 每个字符串由一个字母后跟一个数字组成。

3. L^4 是所有 4 个字母字符串的集合。

4. L^* 是所有由字母组成的串的集合, 包括空串。

5. $L(L \cup D)^*$ 是所有以字母开头的字母和数字字符串的集合。

6. D^+ 是由一个或多个数位构成的字符串的集合。

$L(L \cup D)^*$: 字母数字组合包括空 (字母在前)

D^+ : 数字组合, 不含空

Regular Expressions

Rules that define regexps over an alphabet Σ :

- BASIS: two rules form the basis:
 - ϵ is a regexp, $L(\epsilon) = \{\epsilon\}$
 - If a is a symbol in Σ , then a is a regexp, and $L(a) = \{a\}$
- INDUCTION: Suppose r and s are regexps denoting the languages $L(r)$ and $L(s)$
 - $(r) | (s)$ is a regexp denoting the language $L(r) \cup L(s)$
 - $(r)(s)$ is a regexp denoting the language $L(r)L(s)$
 - $(r)^*$ is a regexp denoting $(L(r))^*$
 - (r) is a regexp denoting $L(r)$. Additional parentheses do not change the language an expression denotes.

在例 3.3 中，我们可以首先给字母和数位集合命名，并使用语言运算符并、连接和闭包这些运算符来描述标识符。这个处理方法非常有用，以至于，人们常常使用一种称为正则表达式的表示方法来描述语言。正则表达式可以描述所有通过对某个字母表上的符号应用这些运算符而得到的语言。在这种表示法中，如果使用 `letter_` 代表任一字母或下划线，并且 `digit_` 来表示数位，那么我们可以通过以下方式描述 C 标识符的语言：

`Letter_ (letter_ | digit) *`

上面的竖线表示并运算，括号用于对子表达式进行分组，星号表示“零个或多个”括号中表达式的连接，将 `letter_` 和表达式其余部分并列表示连接。

正则表达式是使用下面描述的规则，从较小的正则表达式递归构建的。每个正则表达式 r 表示一种语言 $L(r)$ ，它也是根据 r 的子表达式所表示的语言递归定义的。以下是定义某些字母表上的正则表达式以及这些表达式表示的语言的规则。

归纳基础：如下两条规则构成归纳基础：

1. 是正则表达式， $L()$ 是 $\{g\}$ ，即该语言只包含空字符串。

2. 如果 a 是 Σ 中的一个符号, 则 a 是一个正则表达式, 且 $L(a) = \{a\}$, 即这个语言仅包含一个长度为 1 的字符串。请注意, 按照惯例, 我们使用斜体表示符号, 使用粗体表示相应的正则表达式。

归纳步骤:, 由较小的正则表达式构建较大的正则表达式的归纳步骤分四个部分。假设 r 和 s 是分别正则表达式, 表示语言 $L(r)$ 和 $L(s)$ 的。

1. $(r)|(s)$ 是正则表达式, 表示语言 $L(r) \cup L(s)$ 的。

2. $(r)(s)$ 是正则表达式, 表示语言 $L(r)L(s)$ 的。

3. $(r)^*$ 是正则表达式, 表示 $(L(r))^*$ 的。

4. (r) 是表示 $L(r)$ 的正则表达式。最后一条规则表明我们可以在表达式两边添加额外的括号对并不影响表达式所表示的语言。

Regular Expressions Cont.

- Following the rules, regexps often contain **unnecessary pairs of parentheses**. We may drop some if we adopt the conventions:
 - **Precedence**: **closure**^{*} > **concatenation** > **union** |
 - **Associativity**: All three operators are **left associative**, meaning that operations are grouped from the left, e.g.,
 $a | b | c$ would be interpreted as $(a | b) | c$
- Example: $(a) | ((b)^*(c)) = a | b^*c$

正如所定义的, 正则表达式通常包含一些不必要的括号对。如果我们采用以下约定, 则可以删除某些括号对:

- 一元运算符 $*$ 具有最高优先级并且是左结合的。
- 连接具有第二高优先级并且是左结合的
- $|$ 具有最低优先级并且是左结合的。

例如，根据这些约定，我们可以将正则表达式 $(a)|((b)^*(c))$ 替换为 $a|b^*c$ 。两个表达式都表示同一组字符串，这些字符串要么是单个 a ，要么是零个或多个 b 后跟一个 c 。

Regular Expressions Cont.

- Examples: Let $\Sigma = \{a, b\}$
 - $a|b$ denotes the language $\{a, b\}$
 - $(a|b)(a|b)$ denotes $\{aa, ab, ba, bb\}$
 - a^* denotes $\{\epsilon, a, aa, aaa, \dots\}$
 - $(a|b)^*$ denotes the set of all strings consisting of 0 or more a 's or b 's: $\{\epsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$
 - $a|a^*b$ denotes the string a and all strings consisting of 0 or more a 's and ending in b : $\{a, b, ab, aab, aaab, \dots\}$

举例说明：假设 $\Sigma = \{a, b\}$

- $a|b$ 表示语言 $\{a, b\}$
- $(a|b)(a|b)$ 表示语言 $\{aa, ab, ba, bb\}$ ，字母表上长度为 2 的所有字符串的集合。同一语言的另一个正则表达式是 $aa|ab|ba|bb$ 。
- a^* 表示由零个或多个 a 或 b 的实例构成的串的集合，即 $\{\epsilon, a, aa, aaa, \dots\}$ 。
- $(a|b)^*$ 表示由零个或多个 a 或 b 实例组成的所有字符串的集合，即 a 和 b 的构成的所有字符串的集合 $\{\epsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$ 。同一语言的另一个正则表达式是 $(a|b)^*$ 。
- $a|a^*b$ 表示语言 $\{a; b; ab; aab; aaab; \dots\}$ ，即字符串 a 以及所有由零个或多个 a 组成并以 b 结尾的字符串集合。

Regular Expressions Cont.

- A **regular language** (正则语言) is a language that can be defined by a regexp
- If two regexps r and s denote the same language, they are **equivalent**, written as $r = s$
- Each **algebraic law** below asserts that expressions of two different forms are equivalent

LAW	DESCRIPTION
$r s = s r$	$ $ is commutative
$r (s t) = (r s) t$	$ $ is associative
$r(st) = (rs)t$	Concatenation is associative
$r(s t) = rs rt; (s t)r = sr tr$	Concatenation distributes over $ $
$\epsilon r = r\epsilon = r$	ϵ is the identity for concatenation
$r^* = (r \epsilon)^*$	ϵ is guaranteed in a closure
$r^{**} = r^*$	$*$ is idempotent

$(a|b)(a|b)$
 $=$
 $aa|ab|ba|bb$
 Is it true???

可以用正则表达式定义的语言称为正则集。

如果两个正则表达式 r 和 s 表示同一个正则集，我们说它们是等价的，写成 $r = s$ 。例

如， $(a|b) = (b|a)$ 。

正则表达式遵循一些代数定律；每条定律都断言两种不同形式的表达是等价的。图

3.7 显示了一些适用于任意正则表达式 r 、 s 和 t 的代数定律。

$(a|b)(a|b) = aa|ab|ba|bb$ Is it true??? 是正确的

Regular Definitions

- For **notational convenience**, we can give names to certain regexps and use those names in subsequent expressions
- If Σ is an alphabet of basic symbols, then a **regular definition** is a sequence of definitions of the form:

$$\begin{aligned}d_1 &\rightarrow r_1 \\d_2 &\rightarrow r_2 \\&\dots \\d_n &\rightarrow r_n\end{aligned}$$

where:

- Each d_i is a new symbol not in Σ and not the same as the other d 's
- Each r_i is a regexp over the alphabet $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$

为了方便表示，我们可能希望给某些正则表达式命名，并在后续表达式中使用这些名称，就好像这些名称本身就是符号一样。如果 Σ 是基本符号的字母表，则正则定义是具有如下形式的定义序列：

$$\begin{aligned}d_1 &\rightarrow r_1 \\d_2 &\rightarrow r_2 \\&\dots \\d_n &\rightarrow r_n\end{aligned}$$

其中：

1. 每个 d_i 是一个新符号，它们都不在 Σ 中，并且各不相同；
2. 每个 r_i 是字母表 Σ 上的正则表达式，其中只包含 Σ 中的符号和在它之前定义的所有 d 。

我们限制每个 r_i 中只含有 Σ 中的符号和在它之前定义的所有 d ，可以避免递归定义问题，并且我们可以为每个 r_i 单独构造出只包含 Σ 中符号的正则表达式。为此，我们首先将 r_2 （它不能使用 d_1 之外的任何 d ）中的 d_1 替换为 r_1 ，然后再将 r_3 中的 d_1 和 d_2 替换为 r_1 和（替换之后的） r_2 ，依此类推。最后，将 r_n 中的 d_i （ $i = 1; 2; \dots; n-1$ ）替换为 r_i 的经替换后的版本，在这些版本中都只包含 Σ 中的符号。

Regular Definitions - Examples

- **Regexp** for **C identifiers**

letter_ → A | B | ... | Z | a | b | ... | z | _
digit → 0 | 1 | ... | 9
id → *letter_* (*letter_* | *digit*)*

_hello valid?
3times valid?

- **Regexp** for **C identifiers**

$(A|B|...|Z|a|b|...|z|_)((A|B|...|Z|a|b|...|z|_)|(0|1|...|9))^*$

例 3.5: C 标识符是由字母、数字和下划线组成的字符串。这是 C 标识符语言的常规定义。我们将按照惯例使用斜体来表示常规定义中定义的符号。

Extensions of Regular Expressions

- **Basic operators**: union |, concatenation, and Kleene closure * (proposed by Kleene in 1950s)

- A few **notational extensions**:

- **One of more instances**: the unary, postfix operator +
 - $r^+ = rr^*$, $r^* = r^+ | \epsilon$
- **Zero or one instance**: the unary postfix operator ?
 - $r? = r | \epsilon$
- **Character classes**: shorthand for a logical sequence
 - $[a_1a_2...a_n] = a_1 | a_2 | ... | a_n$
 - $[a-e] = a | b | c | d | e$

- The extensions are **only for notational convenience**, they do not change the descriptive power of regexps

自从 Kleene 在 20 世纪 50 年代引入了具有基本运算符并、连接和 Kleene 闭包的正则表达式以来，正则表达式中添加了许多扩展，以增强正则表达式描述字符串模式的能力。在这里，我们介绍一些最早出现在像 lex 这样的 Unix 实用程序中的扩展表示法。这些扩展标识法在词法分析器的规约中特别有用。本章的参考文献包含对当今仍在使用的一些正则表达式变体的讨论。

1. 一个或多个实例。单目后缀运算符 $+$ 表示一个正则表达式及其语言的正闭包。也就是说，如果 r 是正则表达式，则 $(r)^+$ 表示语言 $(L(r))^+$ 。运算符 $+$ 与运算符 $*$ 具有相同的优先级和结合性。两个有用的代数定律 $r^* = r + r^*$ 和 $r + r^* = r^*$ 说明了克林闭包和正闭包之间的关系。

2. 零个或一个实例。单目后缀运算符 $?$ 表示 “零或一次出现。” 也就是说， $r?$ 等价于 $r| \epsilon$ ，或者换句话说， $L(r?) = L(r) \cup \{\epsilon\}$ 。 $?$ 运算符与 $+$ 运算符具有相同的优先级和结合性。

3. 字符类。正则表达式 $a_1|a_2|\dots|a_n$ ，其中 a_i 是字母表的各个符号，可以用简写形式 $[a_1 a_2 a_n]$ 替换。更重要的是，当 $a_1; a_2; \dots; a_n$ 构成逻辑上连续的序列时，例如连续的大写字母、小写字母或数字，我们可以用 $a_1 - a_n$ 来替换它们，即只写出第一个和最后一个符号，中间用连词符分隔。因此， $[abc]$ 是 $a|b|c$ 的简写，而 $[a-z]$ 是 $a|b|z$ 的简写。

Recognition of Tokens

- Lexical analyzer examines the input string and finds a prefix that matches one of the tokens
- The first thing when building a lexical analyzer is to define the patterns of tokens using regular definitions
- **A special token: $ws \rightarrow (blank | tab | newline)^+$**
 - When the lexical analyzer recognizes the **whitespace token**, it does not return it to the parser, but rather restart the lexical analysis from the character that follows the whitespace (the next token gets returned to the parser)

在上一节中，我们学习了如何使用正则表达式来表达模式。现在，我们必须研究如何根据各个需要识别的词法单元的模式来构建一段代码。这段代码能够检查输入字符串，并在输入的前缀中找出一个和某个模式匹配的词素。我们的讨论将利用以下运行示例。

此外，我们还可以让词法分析器负责消除空白符，方法是让它识别如下定义词法单元"ws"：

```
ws ! ( Blank | tab | newline ) +
```

这里，blank、tab 和 newline 是用于表示具有同样名字的 ASCII 字符的抽象符号。词法单元 ws 同其他词法单元的不同之处在于，当我们识别到 ws 时，我们并不会将其返回给语法分析器，而是从这个空白之后的字符开始继续进行词法分析。返回给语法分析器的是下一个词法单元。

Example: Patterns and Tokens

<i>digit</i>	→	[0-9]
<i>digits</i>	→	<i>digit</i> ⁺
<i>number</i>	→	<i>digits</i> (. <i>digits</i>)? (E [+-]? <i>digits</i>)?
<i>letter</i>	→	[A-Za-z]
<i>id</i>	→	<i>letter</i> (<i>letter</i> <i>digit</i>)*
<i>if</i>	→	if
<i>then</i>	→	then
<i>else</i>	→	else
<i>relop</i>	→	< > <= >= = <>

Patterns for tokens

LEXEMES	TOKEN NAME	ATTRIBUTE VALUE
Any <i>ws</i>	—	—
if	if	—
then	then	—
else	else	—
Any <i>id</i>	id	Pointer to table entry
Any <i>number</i>	number	Pointer to table entry
<	relop	LT
<=	relop	LE
=	relop	EQ
<>	relop	NE
>	relop	GT
>=	relop	GE

Tokens, their patterns, and attribute values

例 3.8: 图 3.10 的语法片段描述了分支语句和条件表达式的简单形式。此语法类似于 Pascal 语言的语法, 因为 **then** 关键字显式地出现在条件表达式的后面。对于 **relop**, 我们使用 Pascal 或 SQL 等语言的比较运算符, 其中 **=** 表示“等于”, **<>** 表示“不等于”, 因为它呈现了一个有趣的词素结构。

文法的终结符包括 **if**、**then**、**else**、**relop**、**id** 和 **number**, 对于词法分析器来说是词法单元的名称。这些词法单元的模式使用正则定义进行描述, 如图 3.11 所示。**id** 和 **number** 的模式与示例 3.7 中看到的类似。

对于这种语言, 词法分析器将识别关键字 **if**、**then** 和 **else**, 以及与 **relop**、**id** 和 **number** 模式匹配的词素。为了简化问题, 我们做出一个如下常见的假设: 关键字也是保留字: 也就是说, 它们不是标识符, 虽然它们的词素与标识符的模式匹配。

我们对词法分析器的目标总结在图 3.12 中。该表显示, 对于各个词素或词素的集合, 应该将哪个词法单元名返回到语法分析器以及应该返回什么属性值, 如第 3.1.3 节中讨论的。请注意, 对于其中的六个关系运算符, 符号常量 LT、LE 等被当作属性值返回, 其目的是指明我们发现的是词法单元 **relop** 的哪个实例。找到的运算符将影响编译器输出的代码。

Transition Diagrams (状态转换图)

- An important step in constructing a lexical analyzer is to convert patterns into “**transition diagrams**”
- Transition diagrams have a collection of nodes, called **states** (状态) and **edges** (边) directed from one node to another

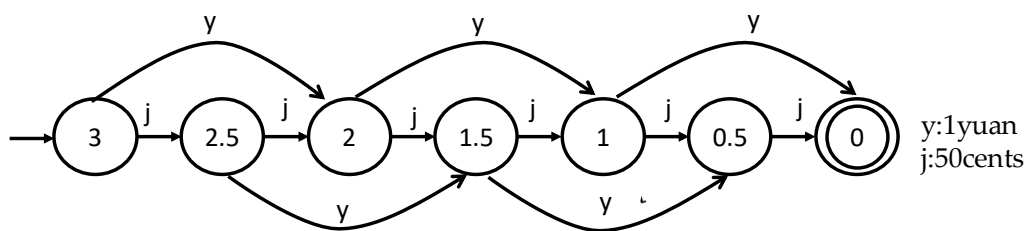
作为构建词法分析器的中间步骤，我们首先将模式转换为特定风格的流图，称为“状态转换图”。在本节中，我们手动执行从正则表达式模式到转换图的转换，但是在第 3.6 节中，我们将看到有一种自动化的方法可以从正则表达式的集合构造状态转换图。

状态转移图有一组被称为“状态”的结点或圆圈。词法分析器在扫描输入串的过程中寻找和某个模式匹配的词素，而转换图中的每个状态代表一个可能在该过程中出现的情况。我们可以将一个状态视为对我们已经看到的位于 `lexemeBegin` 指针和 `forward` 指针之间的字符的总结，它包含了我们在进行词法分析时需要的全部信息（如在图 3.3 的情况下）。

转换图的边从图的一种状态指向另一种状态。每条边的标号都包含了一个符号或多个符号。如果我们处于某种状态 `s`，并且下一个输入符号是 `a`，我们会寻找一条从 `s` 离开且标号为 `a` 的边（该边的标号中可能还包括其他符号）。如果我们找到这样的一条边，我们就将 `forward` 指针前移并进入转移图中该边所指的状态。我们将假设我们所有的转换图都是确定性的，这意味着在对于任何一个给定的状态和任何一个给定的符号，最多只有一个边从该状态离开的边的标号包含该符号。从第 3.5 节开始，我们将放宽对确定性的要求，使词法分析器的设计者更加容易完成任务，但同时提高了对实现者的技巧要求。

What is the process of operating a vending machine?

- Suppose a vending machine can only accept 50 cents and 1 yuan coins. How many states are there for the vending machine to purchase a \$3 bottle of Coke?



有限状态系统

状态：将事物进行区分的标识

具有离散状态的系统：数字电路(0 和 1)，交通信号灯（离散状态系统的状态是有限的）

具有连续状态的系统：水库水位变化，温度变化，具有无穷多个状态

有限状态系统是离散状态的系统

自动售货机的操作流程？如购买一瓶价格为 3 元的可乐需要做哪些步骤？

（虽然现在是用支付宝或者微信支付。。。）

假设某一台售货机只可接受 5 角和 1 元的硬币

购买一瓶 3 元的可乐，售货机的状态有几种？(状态可理解为购物的步骤)

假设已投入 2 元，为了买到可乐，还需投入 1 个 1 元硬币或 2 个 5 角硬币

购买一瓶 3 元的可乐，售货机的状态有几种？

需要 3 元: 还没有投入任何硬币的机器状态

需要 2.5 元: 已投入 1 个 5 角硬币后的状态

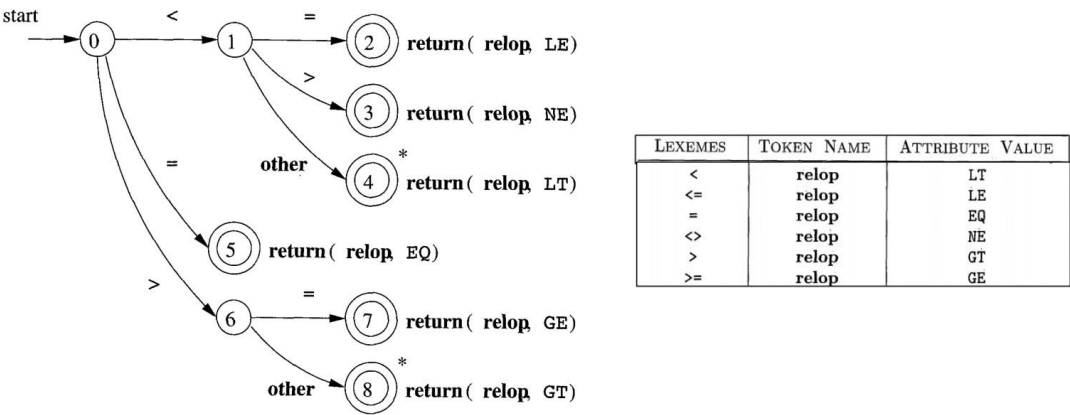
需要 2.0 元: 已投入 2 个 5 角或 1 个 1 元的硬币后的状态

需要 1.5 元: 已投入 3 个 5 角或 1 个 1 元和 1 个 5 角硬币后的状态

需要 1 元: 已投入 4 个 5 角或 1 个 1 元和 2 个 5 角的硬币后的状态

需要 0.5 元: 已投入 5 个 5 角或 3 个 5 角和 1 个 1 元或 1 个 5 角和 2 个 1 元硬币后的状态

需要 0 元: 表示已投入至少 3 元的硬币，可以购买可乐了

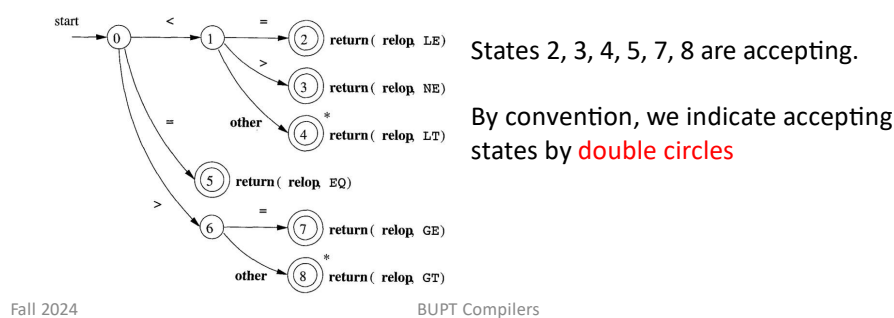


The transition diagram in the left is for recognizing **relop** tokens

例 3.9：图 3.13 是识别所有与词法单元 relop 匹配的词素的状态转换图。

States

- Represent conditions that could occur during the process of scanning the input for recognizing lexemes of tokens (what characters we have seen)
- The **start state** (开始状态), or initial state, is indicated by an edge labeled “start”, which enters from nowhere
- Certain states are said to be **accepting** (接受状态), or final, indicating that a lexeme has been found



Fall 2024

BUPT Compilers

31

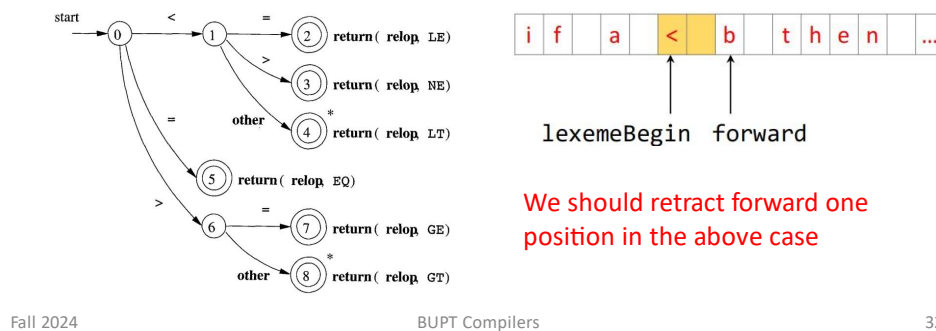
1. 某些状态称为接受状态或最终状态。这些状态表明已找到了一个词素，尽管实际的词素可能并不包含 lexemeBegin 指针和 forward 指针之间的所有字符。我们一直用双圆圈表示一个接受状态，并且如果该状态要执行一个动作的话——通常会向语法分析器返回一个词法单元和相关的属性值——我们将把该行动附加到该接受状态上。

2. 另外，如果需要将 forward 指针回退一个位置（即相应的词素并不包含那个在最后一步使我们到达接受状态的符号），那么我们将在该接受状态的附近加上*。在我们的示例中，不需要将 forward 指针回退多个位置，但如果需要，我们可以将接受状态附加相应数目的*。

3. 指定一个状态为开始状态，或初始状态；该状态由一条没有出发结点的、标号为“start”，的边指明。在读入任何输入符号之前，状态转换图总是从开始状态开始。

States Cont.

- At certain accepting states, the found lexeme may not contain all characters that we have seen from the start state
- Such states are annotated with *
- When entering * states, it is necessary to **retract (回退)** the forward pointer, which points to the next char in the input string)

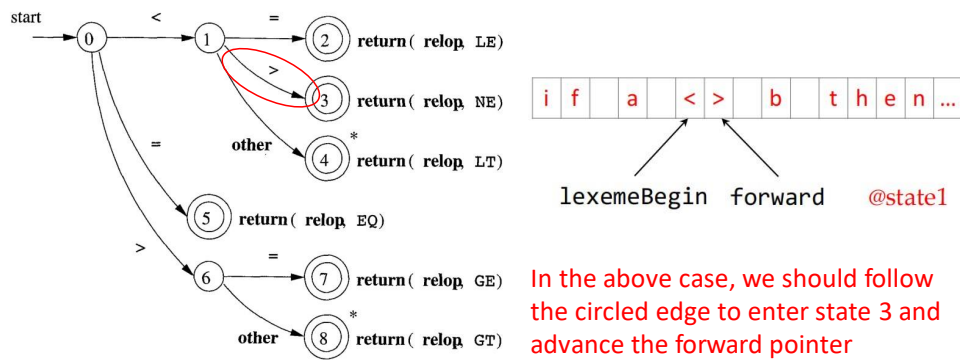


我们从初始状态 0 开始，即起始状态。如果我们看到的第一个输入符号是 <，那么在所有与 relop 模式匹配的词条中，我们只能选择 <、<> 或 <=。因此，我们进入状态 1，并查看下一个字符。如果是=，那么我们识别出词条<=，进入状态 2，并返回带有属性值为 LE 的 relop 词法单元。其中符号常量 LE 代表了这个具体的比较运算符。如果在状态 1 中下一个字符是 >，则我们得到词条 <>，从而进入状态 3 并返回一个词法单元，表明已经找到一个不等于运算符。对于任何其他字符，识别得到的词条为 <，我们进入状态 4 并向语法分析器返回这个信息。但请注意，状态 4 具有一个 *号，说明我们必须将输入回退一个位置。

另一方面，如果在状态 0 中我们看到的第一个字符是 =，那么这个字符一定是要识别的词条。我们立即从状态 5 返回该信息。其余的可能性是第一个字符是 >的情况。那么，我们应该进入状态 6，并根据下一个字符决定词条是 >=（如果我们接下来看到的符号为=），还是只是 >（对于任何其他字符）。请注意，在状态 0 中，如果我们看到的是不同于 <、=、或>的任何字符，我们就不可能看到一个 relop 词条，因此这个状态转换图将不会被使用。

Edges

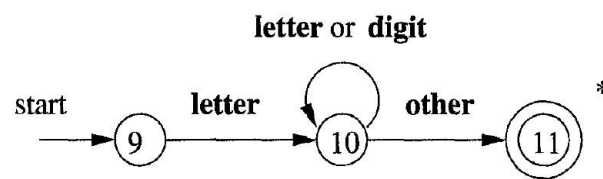
- **Edges** are directed from one state to another
- Each edge is labeled by a symbol or set of symbols



- 边从一个状态指向另一个状态
- 每条边都由一个或一组符号标记

Recognition of Reserved Words and Identifiers (保留字和标识符的识别)

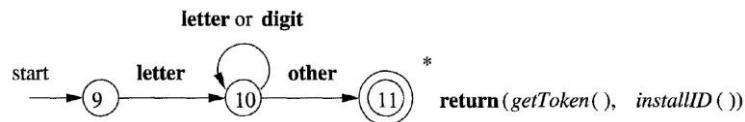
- In many languages, **reserved** words or **keywords** (e.g., `then`) also match the pattern of identifiers
- **Problem:** the transition diagram that searches for identifiers can also recognize reserved words



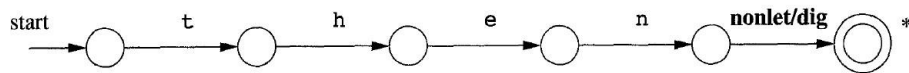
识别关键字和标识符时有一个问题要解决。通常，像 `if` 或 `then` 这样的关键字是被保留的（正如它们在我们运行的示例中一样），因此它们不是标识符，即使它们看起来像标识符。因此，虽然我们通常使用如图 3.14 所示的转换图来搜索标识符词素，但该图也可以将识别出连续使用的例子中的关键字 `if`、`then` 和 `else`。

Handling Reserved Words

- **Strategy 1:** Preinstall the reserved words in the symbol table. Put a field in the symbol-table entries to indicate that these strings are not ordinary identifiers



- **Strategy 2:** Create a separate transition diagram with a high priority for each keyword



有两种方法可以处理看起来像标识符的保留字：

1. 首先将保留字安装到符号表中。符号表条目的字段指示这些字符串并不是普通标识符，并指出它们所代表的词法单元。我们假设图 3.14 中使用的是这种方法。当我们找到一个标识符时，对 `installID` 的调用会将其放入符号表（如果符号表尚不存在），并返回一个指针，指向这个刚找到的词素所对应的符号表条目。当然，任何在词法分析时不在符号表中的标识符都不可能是一个保留字，因此它的词法单元是 `id`。函数 `getToken` 检查对应于刚找到的词素的符号表条目，并根据符号表中的信息返回该词素所代表的词法单元名称——要么是 `id`，要么是一个在初始化时就被加入到符号表中的关键字词法单元。

2. 为每个关键词创建单独的转换图；关键字 `then` 的示例如图 3.15 所示。请注意，这样的转换图包含的状态表示看到该关键字的各个后续字母后的情况，最后是一个“非字母或数字”的测试，也就是检查后面是否为某个不可能成为标识符一部分的字符。有必要检查标识符是否已经结束，否则在碰到词素像 `thenextvalue` 这样以 `then` 为前缀的 `id` 词法单元时，我们可能会错误地返回词法单元 `then`。如果我们采用这种方法，我们必须设定词法单元之间的优先级，使得当一个词素同时匹配 `id` 的模式和关键字的模式时，优先识别保留字

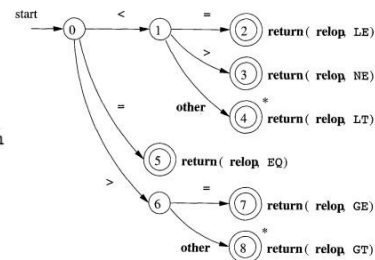
词法单元，而不是 id 词法单元。我们在示例中没有使用这种方法，这也是我们没有对图 3.15 中的状态进行编号的原因。

Building a Lexical Analyzer from Transition Diagrams

```

TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
               or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}

```

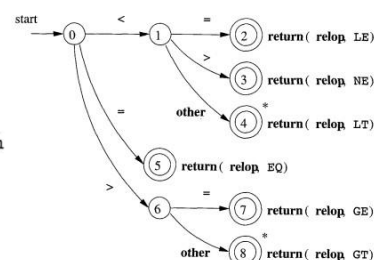


Building a Lexical Analyzer from Transition Diagrams

```

TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
               or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}

```



Use a variable state to record the current state

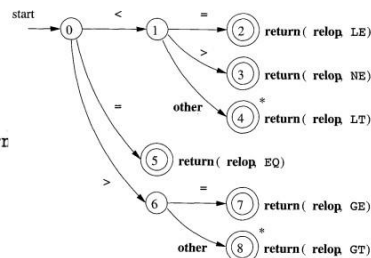
Building a Lexical Analyzer from Transition Diagrams

```

TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
               or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}

```

Sketch of implementation of relop transition diagram



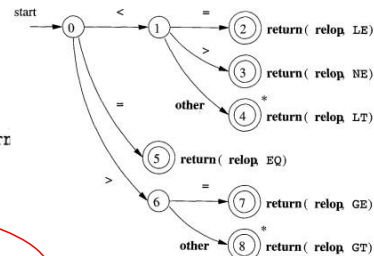
A switch statement based on the value of state takes us to code for each of the possible states

有一个 switch 语句根据 state 的值将我们转到对应于各个可能状态的相应代码段，我们可以在那里找到该状态需要执行的动作。一个状态的代码本身通常也是一个 switch 语句或多路分支，这个语句读取和检查下一个输入字符，由此确定下一个状态。

Building a Lexical Analyzer from Transition Diagrams

```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
               or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```

Sketch of implementation of relop transition diagram



The code of a normal state:

1. Read the next character
2. Determine the next state
3. If step 2 fails, do error recovery

示例 3.10: 在图 3.18 中, 我们可以看到 `getRelop()` 方法的一个概述, 它是一个 C++ 函数, 其作用是模拟图 3.13 的转换图并返回一个 `TOKEN` 类型的对象, 该对象由词法单元名称 (在这种情况下必定是 `relop`) 和一个属性值 (在这种情况下是六个比较运算符之一的编码)。 `getRelop()` 首先创建一个新对象 `retToken` 并将该对象的第一个分量初始化为 `RELOP`, 即词法单元 `relop` 的编码。

我们看到 `case 0` 中一个典型的状态行为。函数 `nextChar()` 从输入中获取下一个字符并将其分配给局部变量 `c`。然后, 我们检查 `c` 中是否有我们期望找到的三个字符, 并在每种情况下使根据图 3.13 所示的状态转换图完成状态转换。例如, 如果下一个输入字符是 `=`, 则转换到状态 5。

如果下一个输入字符不是某个比较运算符的首字符, 则 `getRelop()` 会调用函数 `fail()`。 `fail()` 的具体操作依赖于词法分析器的全局错误恢复策略。它应该将 `forward` 指针重置为 `lexemeBegin` 的值, 使得我们可以使用另一个转换图从尚未处理的输入部分的真正开始位置开始识别。然后, 它可能还会将变量 `state` 值更改为另一个转换图的初始状态, 该转换图将搜索另一个词法单元。在另一种情况下, 如果所有的转换图都已经用过, 则 `fail()` 可以启动一个错误纠错步骤, 如第 3.1.4 节中所述的方法来纠正输入并找到一个词素。

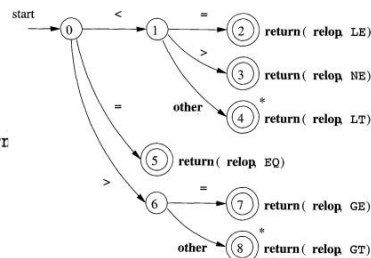
Building a Lexical Analyzer from Transition Diagrams

```

TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
                or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}

```

Sketch of implementation of relop transition diagram



The code of an accepting state:

1. Perform retraction if the state has *
2. Set token attribute values
3. Return the token to parser

我们还在图 3.18 中展示了状态 8 的行为。因为状态 8 带有一个 *号，所以我们必须将输入指针回退一个位置（即，将 c 放回输入流）。该任务是由函数 `retract()` 完成的。由于状态 8 代表了对词素 `>` 的识别，我们把返回对象中的第二个分量（我们假设这个分量的名字是 `attribute`）设置为 `GT`，即该运算符的编码。

Building the Entire Lexical Analyzer

- **Strategy 1:** Try the transition diagram for each token sequentially
 - fail() resets the pointer forward and starts the next diagram
- **Strategy 2:** Run transition diagrams in parallel
 - Need to resolve the case where one diagram finds a lexeme and others are still able to process input.
 - **Solution:** take the longest prefix of the input that matches any pattern
- **Strategy 3: Combining all transition diagrams into one (preferred)**
 - Allow the transition diagram to read input until there is no possible next state
 - Take the longest lexeme that matched any pattern

为了在适当的地方模拟适当的状态转换图，我们考虑几种将如图 3.18 所示的代码集成到整个词法分析器中的方法。

1.我们可以让词法分析器顺序尝试各个词法单元的转换图。然后，在每次调用例 3.10 中的函数 fail()时，它重置 forward 指针并启动下一个转换图。这种方法使我们可以像图 3-15 中建议的那样，为各个关键字使用各自的状态转换图。我们只需在使用 id 的状态转换图之前使用这些关键字的转换图，就可以使得关键字被识别为保留字。

2.我们可以并行运行各种转换图，“将下一个输入字符提供给所有转换图，并允许每个转换图做出它应执行的转换。如果我们采用这一策略，我们必须小心解决如下问题：一个状态转换图已经找到了一个与它的模式相匹配的词素，但另外的一个或多个状态转换图仍然可以继续处理输入。解决这个问题的常见策略是取最长的和某个模式相匹配的输入前缀。该规则允许我们识别出 thennext 而不是关键字 then，识别出 -> 而不是 -，例如。

3. 有一个更好的方法，也是我们将在以下各节中采用的方法，是将所有转换图合并为一个。我们允许合并后的转换图尽量读取输入，直到不存在下一个状态为止；然后像上面的第 (2) 项中讨论的那样取最长的和某个模式匹配的最长词素。在我们的运行示例中，进

行这种合并很简单，因为没有两个词法单元以相同的字符开头；即，根据第一个字符就可以知道我们正在寻找的是哪个词法单元。因此，我们可以直接将状态 0、9、12 和 22 合并成一个开始状态，而保持其他转换不变。然而，一般来说，正如我们不久将看到的那样，合并几个词法单元的转换图的问题更为复杂。

The Lexical-Analyzer Generator Lex

- Lex, or a more recent tool Flex, allows one to specify a lexical analyzer by specifying regexps to describe patterns for tokens
- Often used with Yacc/Bison to create the frontend of compiler

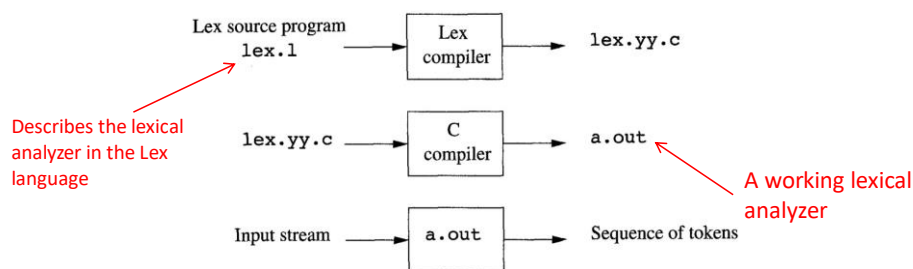


Figure 3.22: Creating a lexical analyzer with Lex

Lex 在本节中，我们将介绍一个名为 Lex 的工具，在最近的实现中它也称为 Flex。它支持使用正则表达式来描述词法单元的模式，由此给出一个词法分析器的归约。Lex 工具的输入表示方法为 Lex 语言，而工具本身则称为 Lex 编译器。在它的核心部分，Lex 编译器将输入的模式转换为一个状态转换图，并生成相应的实现代码，并存放到名为 lex.yy.c 的文件。这些代码模拟了状态转换图。

图 3.22 显示了 Lex 的使用方式。首先，用 Lex 语言编写一个输入文件（我们称之为 lex.l），描述要生成的词法分析器。然后，Lex 编译器将 lex.l 转换为 C 程序，该文件始终命名为 lex.yy.c。最后，文件 lex.yy.c 总是被 C 编译器编译为一个名为 a.out 的文件。C 编译器的输出是一个读取输入字符流并生成词法单元流工作的可允许的词法分析器。

编译后的 C 程序（在图 3.22 中称为 a.out）的通常是一个被语法分析器调用的子例程。这个子例程返回一个整数值，即可能出现的某个词法单元名的编码。而词法单元的属

性值，无论是一个数字编码、还是一个指向符号表的指针，或者什么都没有，都被放置在全局变量 `yylval` 中，该变量由词法分析器和语法分析器共享，从而可以轻松返回一个词法单元名字和一个属性值。

Structure of Lex Programs

- **A Lex program has three sections separated by `%%`**
 - **Declaration (声明)**
 - Variables, manifest constants (e.g., token names)
 - Regular definitions
 - **Translation rules (转换规则) in the form “Pattern {Action}”**
 - Each **pattern (模式)** is a regexp (may use the regular definitions of the declaration section)
 - **Actions (动作)** are fragments of code, typically in C, which are executed when the pattern is matched
 - **Auxiliary functions section (辅助函数)**
 - Additional functions that can be used in the actions

Fall 2024

BUPT Compilers

44

- 声明部分包括变量声明、明示常量（manifest constant，被声明的表示一个常数的标识符，例如，一个词法单元的名字）的声明和 3.3.4 节中描述的正则定义。

- Lex 程序的每个翻译规则的形式为模式{ 动作 }

其中，每个模式都是一个正则表达式，它可以使用声明部分中给出的正则定义。这些动作是代码片段，虽然人们已经创建了许多使用其他语言的 Lex 变体，但这些代码片段通常用 C 语言编写。

- 第三部分包含各个动作需要使用的所有辅助函数。还有一种方法是将这些函数可以单独编译，并与词法分析器的代码一起加载。

Lex Program Example

```
%{  
    /* definitions of manifest constants  
    LT, LE, EQ, NE, GT, GE,  
    IF, THEN, ELSE, ID, NUMBER, RELOP */  
}%  
  
/* regular definitions */  
delim    [ \t\n]  
ws        {delim}+  
letter    [A-Za-z]  
digit     [0-9]  
id         {letter}{(letter){digit}}*  
number     {digit}+(\.{digit})+(E[+-]?{digit})?
```

Anything in between %{ and }% is copied directly to lex.yy.c.
In the example, there is only a comment, not real C code to define manifest constants

Regular definitions that can be used in translation rules

Section separator

Fall 2020

CS323 Compilers

45

例 3.11：图 3.23 是一个 Lex 程序，它能够识别图 3.12 的各个词法单元并返回找到的词法单元。观察这段代码可以发现 Lex 的许多重要特点。

在声明部分，我们看到一对特殊的括号：%{ 和 }%。这些括号内的所有内容都会直接复制到文件 lex.yy.c，它们不会被视作正则定义。通常将明示常量的定义放在该括号里，并使用 C 语言的#define 语句给每个明示常量赋予一个唯一的整数编码。在我们的示例中，我们在注释中列出了 LT、IF 等明示常量，但没有显示它们被赋予哪些特定的整数。

声明部分中还包含一个正则定义的序列。这些定义使用了第 3.3.5 节中描述的正则表达式的扩展表示法。那些将在后面的定义中或某个翻译规则的模式中使用的正则定义用大括号括起来。因此，例如，delim 被定义为表示一个包含了空格、制表符和换行符的字符类的简写；与所有 UNIX 命令一样，后两个字符分别用反斜杠再跟上 t 或 n 表示。这个表示法和 Unix 命令使用的方法相同。于是，ws 通过正则表达式 {delim}+ 定义为一个或多个分隔符组成的序列。

请注意，在 id 和 number 的定义中，圆括号是用于分组的元符号，并不代表其圆括号本身。相反，number 定义中的 E 代表其自身。如果我们希望使用 Lex 的某个元符号（例如何括号、+、* 或? 等）来代表其自身，我们可以在它们前面加上反斜杠。例

如，我们在 `number` 的定义中看到 `\`，就表示小数点本身。在它前面加上反斜线的原因是，和在 UNIX 正则表达式中一样，该字符在 `lex` 是一个代表“任一字符”的元符号。

Lex Program Example Cont.

```
{ws}      /* no action and no return */
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yylval = (int) installID(); return(ID);}
{number}  {yylval = (int) installNum(); return(NUMBER);}
"<"      {yylval = LT; return(RELOP);}
"<="     {yylval = LE; return(RELOP);}
"="       {yylval = EQ; return(RELOP);}
"< >"    {yylval = NE; return(RELOP);}
">"      {yylval = GT; return(RELOP);}
">="     {yylval = GE; return(RELOP);}

%%
```

Continue to recognize other tokens

Return token name to the parser

Place the lexeme found in the symbol table

A global variable that stores a pointer to the symbol table entry for the lexeme. Can be used by the parser or a later component of the compiler.

首先，`ws`（在第一部分中声明的标识符）有一个相关的空动作。如果我们发现了一个空白符，我们并不把它返回给语法分析器，而是继续寻找另一个词素。第二个词法单元具有简单的正则表达式模式 `if`。如果我们在输入中看到两个字母 `if`，并且 `if` 后面没有跟随其他字母或数位（如果有的话，词法分析器会去寻找一个和 `id` 模式匹配的最长输入前缀），那么词法分析器从输入中读入这两个字符，并返回词法单元名 `IF`，即明示常量 `IF` 所代表的整数值。关键字 `then` 和 `else` 的处理方式类似。

第五个词法单元的模式由 `id` 定义的模式。请注意，虽然像 `if` 这样的关键字既匹配此模式也匹配之前的一个模式，但在最长匹配前缀与两个或多个模式匹配的情况下 `Lex` 会选择最先被列出的那个模式。当 `id` 被匹配时，相应的处理动作分为三步：

1. 调用函数 `installID()` 将找到的词素放入符号表中。
2. 该函数返回一个指向符号表的指针，该指针被放置在全局变量 `yylval` 中，可供语法分析器或编译器的后续组件使用。请注意，`installID()` 可以使用以下两个由 `Lex` 生成的、

由词法分析器自动赋值的变量： (a) `ytext` 是指向词素开头的指针，类似于图 3.3 中的 `lexemeBegin`。 (b) `yyleng` 存放刚找到的词素的长度。

3. 词法单元名 ID 返回给语法分析器。

当一个词素与模式 `number` 匹配时，执行的处理与此类似，它使用辅助函数 `installNum()` 完成处理。

Lex Program Example Cont.

- Everything in the auxiliary function section is copied directly to the file `lex.yy.c`
- Auxiliary functions may be used in actions in the translation rules

```
int installID() { /* function to install the lexeme, whose
                  first character is pointed to by ytext,
                  and whose length is yyleng, into the
                  symbol table and return a pointer
                  thereto */
}

int installNum() { /* similar to installID, but puts numer-
                    ical constants into a separate table */
}
```

Variables defined and set automatically by the lexical analyzer Lex generates

在辅助函数部分，我们看到两个这样的函数： `installID()` 和 `installNum()`。和位于 `%{...%}` 中的声明部分一样，出现在辅助部分中的所有内容都被直接复制到文件 `lex.yy.c` 中，虽然它们位于转换规则部分之后，但这些函数可以在规则部分的动作定义中使用。

Conflict Resolution

- When the generated lexical analyzer runs, it analyzes the input looking for **prefixes that match any of its patterns**.^{*}
- **Rule 1:** If it finds multiple such prefixes, it takes the **longest** one
 - The analyzer will treat `<=` as a single lexeme, rather than `<` as one lexeme and `=` as the next
- **Rule 2:** If it finds a prefix matching different patterns, **the pattern listed first** in the Lex program is chosen.
 - If the keyword patterns are listed before identifier pattern, the lexical analyzer will not recognize keywords as identifiers

^{*}See Flex manual for details (Chapter 8: How the input is matched) at <http://dinosaur.compilertools.net/flex/>

我们已经提到了当输入的多个前缀与一种或多种模式匹配时，Lex 用来决定要选择
的正确词素的两条规则：

1. 总是优先选择较长的前缀而不是较短的前缀。
2. 如果最长的可能前缀与两个或多个模式匹配，则优先选择 Lex 程序中首先列出的模式。

Finite Automata (有穷自动机)

- Finite automata are the simplest machines to recognize patterns
- They are essentially graphs like transition diagrams. They simply say “yes” or “no” about each possible input string.
 - Nondeterministic finite automata (NFA, 非确定有穷自动机): A symbol can label several edges out of the same state (allowing multiple target states), and the empty string ϵ is a possible label.
 - Deterministic finite automata (DFA, 确定有穷自动机): For each state and for each symbol of its input alphabet, there is exactly one edge with that symbol leaving that state.
- NFA and DFA recognize the same languages, regular languages, that regexps can describe.

现在我们将了解 Lex 如何将其输入程序转变为词法分析器。转变的核心是被称为有穷自动机的表示方法。这些自动机本质上是与状态转换图类似的图，但有一些区别：

1. 有限自动机是识别器；它们只能对每个可能的输入字符串回答“是”或“否”。
2. 有限自动机有两种形式：

(a) 非确定性有限自动机 (NFA) 对其边的标号没有任何限制。一个符号标记离开同一状态的多条边，并且空字符串，也可以作为标号。

(b) 确定性有限自动机 (DFA) 对于每个状态以及自动机输入字母表中的每个符号都有且只有一条离开该状态，以该符号为标号的边。

确定性和非确定的有穷自动机能识别的语言的集合是相同的。事实上，这些语言正好是能够用正则表达式描述的语言的集合，这个集合中的语言称为正则语言。

Nondeterministic Finite Automata

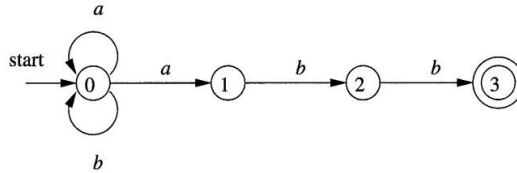
- A nondeterministic finite automaton (NFA) consists of:
 - A finite set of states S
 - A set of input symbols Σ , the **input alphabet**. We assume that the empty string ϵ is never a member of Σ .
 - A **transition function** that gives, for each state, and for each symbol in $\Sigma \cup \{\epsilon\}$ a set of next states.
 - A **start state** (or initial state) s_0 from S
 - A set of **accepting states** (or final states) F , a subset of S

不确定性有限自动机 (NFA) 包括：

1. 有限状态集 S 。
2. 一组输入符号集合，即输入字母表。我们假设代表空字符串的 ϵ 永远不是 Σ 的成员。
3. 一个转换函数，它为每个状态和 $\Sigma \cup \{\epsilon\}$ 中的每个符号都给出了相应的后继状态的集合。
4. S 中的一个状态 s_0 ，被指定为开始状态（或初始状态）。
5. S 的一个子集 F 被指定为接受状态（或者说终止状态）。

NFA Example

- $S = \{0, 1, 2, 3\}$
- Start state: 0
- Accepting states: $\{3\}$
- Transition function



- $(0, a) \rightarrow \{0, 1\}$ $(0, b) \rightarrow \{0\}$
- $(1, b) \rightarrow \{2\}$ $(2, b) \rightarrow \{3\}$

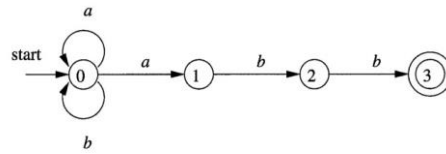
我们可以通过转移图来表示 NFA 或 DFA，图中的结点是状态，带标号的边表示自动机的转移函数。从状态 s 到状态 t 存在一条标号为 a 的边当且仅当状态 t 是状态 s 在输入 a 上的后继状态之一。该图非常类似于转换图，除了：

- 相同的符号可以标记从同一状态出发到达多个目标状态的多条边
- 一条边的标号不仅可以是输入字母表中的符号，也可以是空字符串。

例 3.14： 识别正则表达式 $(a|b)^*abb$ 语言的 NFA 转换图如图 3.24 所示。这个抽象示例描述了所有由 a 和 b 组成的，以字符串 abb 结尾的字符串。这个例子将贯穿本节。虽然它很抽象，但是书籍上它与一些具有实际意义的语言的正则表达式相似。例如，描述所有其名字以 $.o$ 结尾的文件的表达式是 $\text{any}^*.o$ ，其中 any 代表任何可打印字符。

按照我们的转换图约定，状态 3 的双圈表示该状态是接受状态。请注意，从起始状态 0 到接受状态的所有路径都是先在状态 0 上运行一段时间，然后从输入读取 abb ，分别进入状态 1、2 和 3。因此，能够到达接受状态的所有字符串都是以 abb 结尾的字符串。

Transition Table



- Another representation of an NFA
 - **Rows** correspond to states
 - **Columns** correspond to the input symbols or ϵ
 - **The table entry** for a state-input pair lists the set of next states
 - $\emptyset$: the transition function has no info about the state -input pair

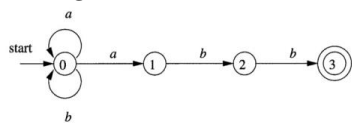
STATE	a	b	ϵ
0	$\{0, 1\}$	$\{0\}$	$\emptyset$
1	$\emptyset$	$\{2\}$	$\emptyset$
2	$\emptyset$	$\{3\}$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

我们还可以通过转换表来表示 NFA，表的各行对应于状态，各列对应于输入符号和 ϵ 。对应于一个给定状态和给定输入的条目是将 NFA 的转换函数应用于这些参数后得到的值。如果转换函数没有给出对应于某个状态-输入对的信息，我们就把 $\emptyset$ 放入相应的表项中。

例 3.15：图 3.24 的 NFA 的转换表如图 3.25 所示。转换表的优点是我们可以轻松确定一个给定状态和一个输入符号相对应的转换。它的缺点是，当输入字母表很大时，且大多数状态在大多数输入字符上没有转换的时候，转换表需要占用大量空间。

Acceptance of Input Strings

- An NFA **accepts** an input string **x** **if and only if**
 - There is a path in the transition graph from the start state to one accepting state, such that the symbols along the path form x (ϵ labels are ignored).



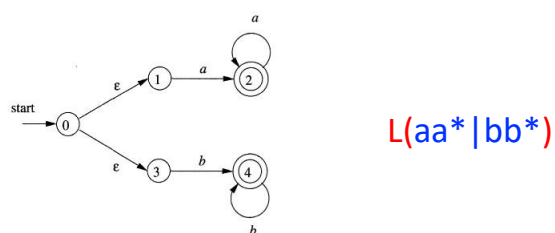
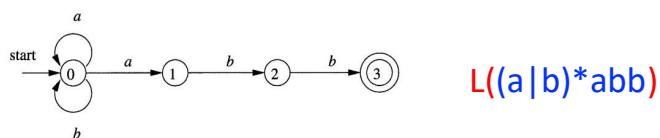
accepts the string *aabb*

- The **language** defined or accepted by an NFA
 - The set of strings labelling some path from the start state to an accepting state

请注意，可能还存在多条具有相同标号序列、但是到达不同状态的路径。例如下面的路径是另一条从状态 0 出发、标号序列同样为 *aabb* 的路径。这条路径最后仍回到状态 0，但状态 0 不是接受状态。然而，请记住，只要存在某条其标号序列为某字符串的路径能够从开始状态达到某个接受状态，NFA 就会接受该字符串。存在某些到达非接受状态的路径，并不会影响这个结论。

由一个 NFA 定义（或接受）的语言是从开始到某个接受状态的所有路径上的标号串的集合。

NFA and Regular Languages



正如前面提到的，图 3.24 的 NFA 定义的语言和正则表达式 $(a | b)^* abb$ 定义的语言相同，即所有来自字母表 $\{a, b\}$ 且以 abb 结尾的串的集合。我们可以用 $L(A)$ 来代表自动机 A 接受的语言。

例 3.17: 图 3.26 是一个接受 $L(aa^* | bb^*)$ 的 NFA。

字符串 aaa 被这个 NFA 接受。请注意，路径中的空标号在连接时消失了，因此这条路径的标签是 aaa 。

Deterministic Finite Automata (DFA)

- A deterministic finite automaton is a special NFA where:
 - There are no moves on input ϵ
 - For each state s and input symbol a , there is exactly one edge out of s labeled a
- DFA can efficiently accept/reject strings (recognize patterns)
- Every regexp and every NFA can be converted to a DFA accepting the same language

确定性有限自动机 (DFA) 是 NFA 的特例，其中：

1. 没有输入 ϵ 上的转换
2. 对于每个状态 s 和输入符号 a ，由且只有一条标号为 a 的边离开 s 。

如果我们使用转换表来表示一个 DFA，那么表中的每个表项就是一个状态。因此，我们可以不使用花括号，直接写出这个状态，因为花括号只是用来说明表项的内容的一个集合。

NFA 抽象地表示了用来识别某种语言中字符串的算法，而相应的 DFA 则是一个简单的识别字符串的算法。

- DFA 可以高效地接受/剔除字符串（识别模式）
- 每个 regexp 和每个 NFA 都可以转换成接受相同语言的 DFA

Simulating a DFA

- **Input:**

- String x terminated by an end-of-file character eof.
- DFA D with start state s_0 , accepting states F , and transition function $move$

- **Output:** Answer “yes” if D accepts x ; “no” otherwise

```
 $s \leftarrow s_0;$   
 $c \leftarrow nextChar();$   
while (  $c \neq eof$  ) {  
     $s \leftarrow move(s, c);$   
     $c \leftarrow nextChar();$   
}  
if (  $s$  is in  $F$  ) return "yes";  
else return "no";
```

算法 3.18: 模拟 DFA。

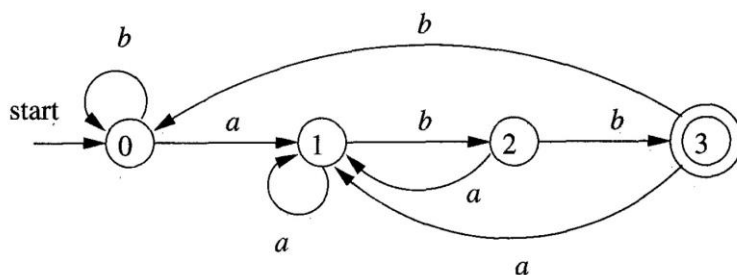
INPUT: 一个以文件结束符 eof 结尾的输入字符串 x 。DFA D 具有起始状态 s_0 、接受状态集 F 和转换函数 $move$ 。

输出: 如果 D 接受 x , 则回答“是”; 否则回答“否”。

方法: 将图 3.27 中的算法应用于输入字符串 x 。函数 $move(s; c)$ 给出了从状态 s 出发, 标号为 c 的边所到达的状态。函数 $nextChar$ 返回输入字符串 x 的下一个字符。

DFA Example

- Given the input string `ababb`, the DFA below enters the sequence of states `0, 1, 2, 1, 2, 3` and returns "yes"



What's the language defined by this DFA?

示例 3.19: 在图 3.28 中显示的一个 DFA 的转换图。该 DFA 接受的语言与与图 3.24 的 NFA 所接受的语言相同, 都是 $(a \mid b)abb$ 。给定输入字符串 `ababb`, 该 DFA 顺序进入状态序列 `0; 1; 2; 1; 2; 3` 并返回 `\yes`。”

From Regular Expressions to Automata

- Regexps concisely & precisely describe the patterns of tokens
- DFA can efficiently recognize patterns (comparatively, the simulation of NFA is less straightforward*)
- When implementing lexical analyzers, regexps are often converted to DFA:
 - $\text{Regexp} \rightarrow \text{NFA} \rightarrow \text{DFA}$

* There may be multiple transitions at a state when seeing a symbol

正则表达式非常适合描述词法分析器和其他模式处理软件。如第 3.5 节所述。然而，该软件的实现需要模拟 DFA（如算法 3.18 所示），或者模拟 NFA 的执行。由于 NFA 对于一个输入符号可以选择不同的转换（如图 3.24 所示状态 0 上输入为 a 时）它可以执行输入 上的转换（如图 3.26 中的状态 0 上时），甚至可以选择是对 或是对真实的输入符号执行转换，因此对 NFA 的模拟不如 对 DFA 的模拟简单。因此，我们需要将一个 NFA 转换为一个识别相同语言的 DFA。

在本节中，我们将首先介绍如何将 NFA 转换为 DFA。然后，我们使用这种称为“子集构造”的技术给出一个直接模拟 NFA 的算法。这个算法可用于那些将 NFA 转化到 DFA 比直接模拟 MFA 更加耗时的（非词法分析的）情形。接着，我们将说明如何把正则表达式转换为 NFA，在必要时可以根据这个 NFA 构造出一个 DFA。最后讨论了不同的正则表达式实现技术之间的时空权衡问题，并说明如何为具体的应用程序选择适当的方法。

Conversion of an NFA to a DFA

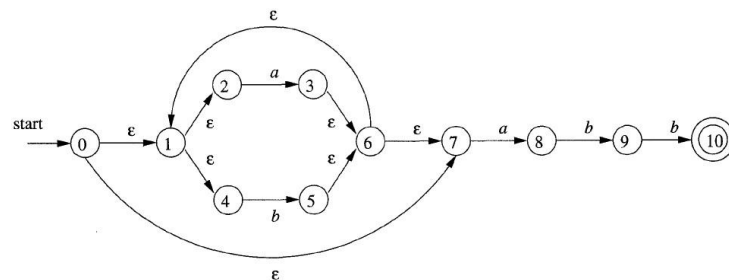
- The subset construction algorithm (子集构造法)
 - **Insight**: Each state of the constructed DFA corresponds to a set of NFA states
 - After reading the input $a_1a_2\dots a_n$, the DFA is in the state which corresponds to the set of states that the NFA can reach, from its start state, following paths labeled $a_1a_2\dots a_n$
 - The algorithm simulates “in parallel” all possible moves an NFA can make on a given input string

子集构造背后的总体思想是让构造得到的 DFA 的每个状态对应于 NFA 的一个状态集合。DFA 在读取输入 $a_1 a_2 a_1 a_2 a_n$ 之后到达的状态对应于相应 NFA 从其起始状态出发，沿着 $a_1 a_2 a_n$ 为标号的路径能够到达的状态集合。

- 算法 “并行” 模拟 NFA 在给定输入字符串上可能做出的所有动作

Example for Algorithm Illustration

- The NFA below accepts the string *babb*
 - There exists a path from the start state 0 to the accepting state 10, on which the labels on the edges form the string *babb*



算法示例

- 下面的 NFA 接受字符串 *babb*

存在一条从起始状态 0 到接受状态 10 的路径，在这条路径上，边上的标签构成了一个字符串 *babb*

Subset Construction Technique

- It is possible that # DFA states is exponential in # NFA states (**worst case**)
 - Each DFA state corresponds to a subset of NFA states
- However, for real languages, the NFA and DFA have **approximately the same number of states**, and the exponential behavior is not seen

DFA 状态数有可能是 NFA 状态数的指数，在这种情况下，我们在试图实现这个 DFA 时会遇到困难。然而，基于自动机的词法分析方法的处理能力部分源于以下事实：对于一个真实语言，它的 NFA 和 DFA 的状态数量大约相同，状态数量呈指数关系的情形尚未在实践中出现过。

Subset Construction Technique Cont.

- Operations used in the algorithm:
 - **ϵ -closure(s)**: Set of NFA states reachable from NFA state s on ϵ -transitions alone
 - **ϵ -closure(T)**: Set of NFA states reachable from some NFA state s in set T on ϵ -transitions alone
 - That is, $\bigcup_{s \in T} \epsilon\text{-closure}(s)$
 - **move(T, a)**: Set of NFA states to which there is a transition on input symbol a from some state s in T

我们的第一个问题是正确处理 N 的空转换。在图 3.31 中，我们看到了几个函数的定义，这些函数描述了一些需要在这个算法中执行的 N 的状态集上的基本计算。请注意， s 表示 N 的单个状态，而 T 代表 N 的一个状态集。

- $\epsilon\text{-closure}(s)$: 仅在 ϵ -transitions 上可从 NFA 状态 s 到达的 NFA 状态集合
 - $\epsilon\text{-closure}(T)$: 仅在 ϵ -transitions 上可从集合 T 中的某个 NFA 状态 s 到达的 NFA 状态的集合
- move (T, a): 输入符号 a 可以从 T 中的某个状态 s 过渡到的 NFA 状态集合

Subset Construction Technique Cont.

- Computing ϵ -closure(T)

- It is a graph traversal process (only consider ϵ edges)

- Computing ϵ -closure(s) is essentially the same

```
push all states of  $T$  onto  $stack$ ;  
initialize  $\epsilon$ -closure( $T$ ) to  $T$ ;  
while (  $stack$  is not empty ) {  
    pop  $t$ , the top element, off  $stack$ ;  
    for ( each state  $u$  with an edge from  $t$  to  $u$  labeled  $\epsilon$  )  
        if (  $u$  is not in  $\epsilon$ -closure( $T$ ) ) {  
            add  $u$  to  $\epsilon$ -closure( $T$ );  
            push  $u$  onto  $stack$ ;  
        }  
}
```

我们必须找出当 N 读入了某个输入串之后可能位于的所有状态集合。作为基础，在读取第一个输入符号之前， N 可以处于 ϵ -closure(s_0) 中的任何状态上，其中 s_0 是 N 的起始状态。下面进行归纳，假设 N 在读取输入字符串 x 后可以处于状态集 T 中的状态上。如果接下来读取输入 a ，则 N 可以立即进入 $move(T; a)$ 中的任何状态。然而， N 可以在读入 a 后再执行几次 ϵ -转换；因此 N 在读入 xa 之后，可以处于 ϵ -closure ($move(T; a)$) 中的任何状态上。

Subset Construction Technique Cont.

- The construction of the set of D 's states, $Dstates$, and the transition function $Dtran$ is also a search process

- Initially, the only state in $Dstates$ is $\epsilon\text{-closure}(s_0)$ and it is unmarked
- Unmarked state means that its next states have not been explored

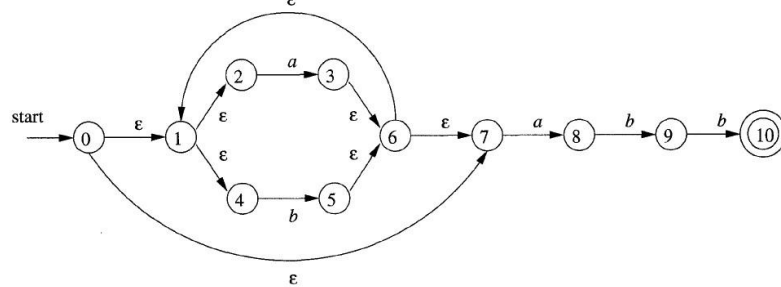
```
while ( there is an unmarked state  $T$  in  $Dstates$  ) {  
    mark  $T$ ;  
    for ( each input symbol  $a$  ) {  
         $U = \epsilon\text{-closure}(\text{move}(T, a))$ ;  
        if (  $U$  is not in  $Dstates$  )  
            add  $U$  as an unmarked state to  $Dstates$ ;  
         $Dtran[T, a] = U$ ;  
    }  
}
```

根据这些想法，我们可以得到如图 3.32 中显示的方法，该方法构造了 D 的状态集 $Dstates$ 和 D 的转换函数 $Dtran$ 。

D 的起始状态是 $\epsilon\text{-closure}(s_0)$ ， D 的接受状态是所有至少包含了 N 的一个接受状态的状态集合。我们只需要说明如何对 NFA 的任何状态集合 T 计算 $\epsilon\text{-closure}(s_0)$ ，就可以完整地描述子集构造法。这个计算过程如图 3.33 所示，它是从一个状态集合开始的一次简单的图搜索过程，不过此时假设这个图只存在标号为空的边。

Example

- A: $\epsilon\text{-closure}(0) = \{0, 1, 2, 4, 7\}$
- B: $Dtran[A, a] = \epsilon\text{-closure}(\{3, 8\}) = \{1, 2, 3, 4, 6, 7, 8\}$
- C: $Dtran[A, b] = \epsilon\text{-closure}(\{5\}) = \{1, 2, 4, 5, 6, 7\}$
- D: $Dtran[B, b] = \epsilon\text{-closure}(\{5, 9\}) = \{1, 2, 4, 5, 6, 7, 9\}$
- ...



例 3.21: 图 3.34 显示了另一个接受 $(a \mid b)^*abb$ 的 NFA; 它恰好是我们在第 3.7 节中根据这个正则表达式直接构造得到的 NFA。让我们将算法 3.20 应用于图 3.34。

等价 DFA 的起始状态 A 为 $\epsilon\text{-closure}(0)$, 即 $A = \{0; 1; 2; 4; 7\}$ 。A 中的状态就是能够从状态 0 出发, 只经过标号为空的路径达到的所有状态。请注意, 因为路径可以包含边, 所以状态 0 也是可从它自身出发经过标号为空的路径到达的状态。

NFA 的输入字母表是 $\{a; b\}$ 。因此, 我们的第一步是标记 A, 并计算 $Dtran[A; a] = \epsilon\text{-closure}(\text{move}(A; a))$ 和 $Dtran[A; b] = \epsilon\text{-closure}(\text{move}(A; b))$ 。在状态 0、1、2、4 和 7 中, 只有 2 和 7 分别到达状态 3 和 8。因此, $\text{move}(A; a) = \{3; 8\}$ 。同时, $\epsilon\text{-closure}(\{3; 8\}) = \{1; 2; 3; 4; 6; 7; 8\}$, 所以我们得出结论

$$Dtran[A; a] = \epsilon\text{-closure}(\text{move}(A; a)) = \epsilon\text{-closure}(\{3; 8\}) = \{1; 2; 3; 4; 6; 7; 8\}$$

我们称这个集合为 B, 得到 $Dtran[A; a] = B$ 。

现在, 我们要计算 $Dtran[A; b]$ 。在 A 中的状态中, 只有 4 有一个输入 b 上的转换, 它到达状态 5。因此,

$$Dtran[A; b] = \epsilon\text{-closure}(\text{move}(A; b)) = \epsilon\text{-closure}(\{5\}) = \{1; 2; 4; 5; 6; 7\}$$

我们称这个集合为 C, 因此 $Dtran[A; b] = C$ 。

如果我们对未加标记的集合 B 和 C 继续这个处理过程，最终会使得这个 DFA 的所有状态都被加上标记。这个结论一定正确，因为 11 个 NFA 状态的集合“仅”有 211 个子集。我们实际构造出 5 个不同的 DFA 状态、它们对应的 NFA 状态集以及 DFA D 的转换表如图所示图 3.35 所示， D 的转换图如图 3.36 所示。状态 A 是 D 的起始状态，而包含 NFA 的状态 10 的状态 E ，是唯一的接受状态。

请注意， 相比 图 3.28 中接受同一语言的 DFA，这个 DFA D 多了一个状态。状态 A 和 C 具有相同的转换函数，因此可以合并。我们将在 3.9.6 节中讨论最小化 DFA 的状态数的问题。

T	T_a	T_b	
$\{0,1,2,4,7\}$ A	$\{1,2,3,4,6,7,8\}$ B	$\{1,2,4,5,6,7\}$	C
$\{1,2,3,4,6,7,8\}$ B	$\{1,2,3,4,6,7,8\}$ B	$\{1,2,4,5,6,7,9\}$	D
$\{1,2,4,5,6,7\}$ C	$\{1,2,3,4,6,7,8\}$ B	$\{1,2,4,5,6,7\}$	C
$\{1,2,4,5,6,7,9\}$ D	$\{1,2,3,4,6,7,8\}$ B	$\{1,2,4,5,6,7,10\}$	E
$\{1,2,4,5,6,7,10\}$ E	$\{1,2,3,4,6,7,8\}$ B	$\{1,2,4,5,6,7\}$	C

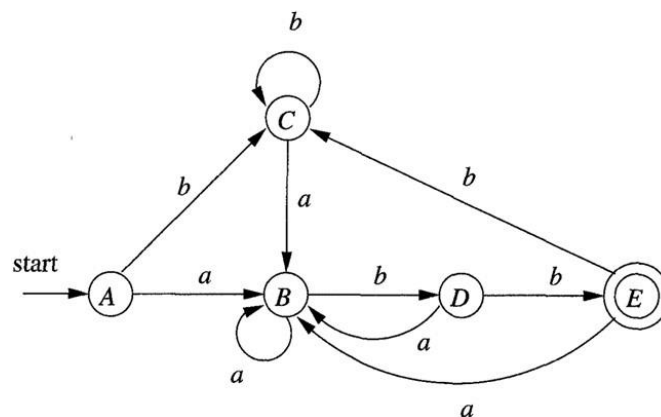
Transition Table of the DFA

- Start state: A; Accepting states: {E}

NFA STATE	DFA STATE	<i>a</i>	<i>b</i>
{0, 1, 2, 4, 7}	<i>A</i>	<i>B</i>	<i>C</i>
{1, 2, 3, 4, 6, 7, 8}	<i>B</i>	<i>B</i>	<i>D</i>
{1, 2, 4, 5, 6, 7}	<i>C</i>	<i>B</i>	<i>C</i>
{1, 2, 4, 5, 6, 7, 9}	<i>D</i>	<i>B</i>	<i>E</i>
{1, 2, 3, 5, 6, 7, 10}	<i>E</i>	<i>B</i>	<i>C</i>

This DFA can be further minimized: A and C have the same moves on all symbols and can be merged.

Transition Graph of the DFA



此 DFA D 识别的语言同样为 $L(D) = \{(a|b)^*abb\}$ ，由此可知构造出来的 DFA D 与 NFA M 是等价的

Regular Expression to NFA

- **Thompson's construction algorithm** (Thompson构造法)
- The algorithm works **recursively** by splitting an expression into its constituent subexpressions, from which the NFA will be constructed using a set of rules
- The rules for constructing an NFA:
 - **Two basis rules (基本规则)**: handle subexpressions with no operators
 - **Three inductive rules (归纳规则)**: construct larger NFA's from the NFA's for subexpressions

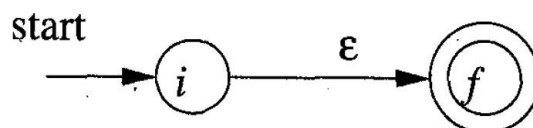
我们现在给出一种算法，用于将任何正则表达式转换为接受相同语言的 NFA。该算法是语法制导的，即它沿着正则表达式的语法分析树自底向上递归地进行处理。对于每个子表达式，算法都会构造一个只有一个接受状态的 NFA。

- 基本规则用于处理不带运算符的子表达式
- 归纳规则则根据一个给定表达式的直接子表达式的 NFA 构造出这个表达式的 NFA 的。

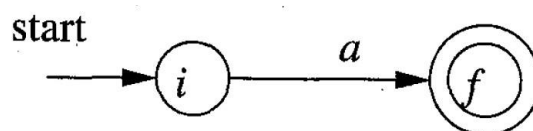
Thompson's Construction Algorithm

Two basis rules:

1. The **empty expression** ϵ is converted to



2. Any subexpression a (a **single symbol** in input alphabet) is converted to



BASIS:

- 对于表达式空构造下面的 NFA

这里, i 是一个新状态, 即该 NFA 的起始状态, f 是另一个新状态, 即该 NFA 的接受状态。

- 对于字母表中的子表达式 a , 构造 下面的 NFA

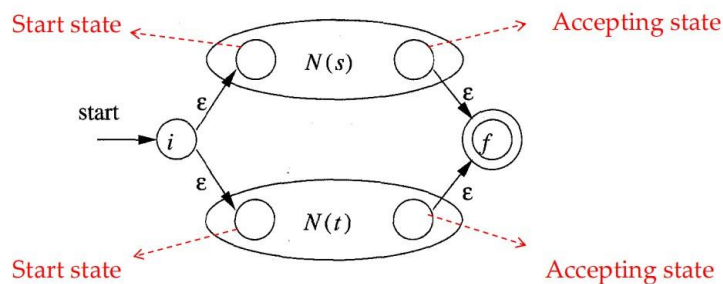
同样 i 和 f 都是新状态, 分别是这个 NFA 的开始状态和接受状态。

请注意, 在这两种基础构造规则中, 对于空或某个 a 的作为 r 的子表达式的每次出现, 我们都会使用新状态分别构造出一个独立的 NFA。

Thompson's Construction Algorithm

The inductive rules: the union case

- $s \mid t$: $N(s)$ and $N(t)$ are NFA's for subexpressions s and t



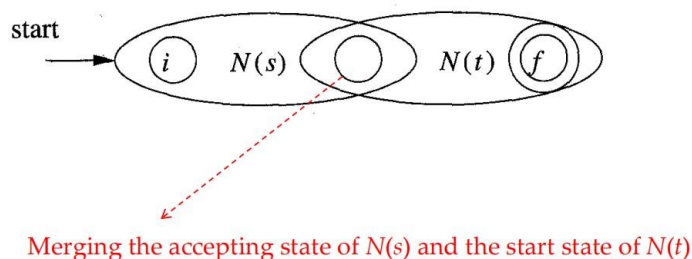
归纳：假设正则表达式 s 和 t 的 NFA 分别为 $N(s)$ 和 $N(t)$ 。

a) 假设 $r = s \mid t$, r 的 NFA, 即 $N(r)$, 如图 3.40 所示构建。这里, i 和 f 是新状态, 分别是 $N(r)$ 的开始状态和接受状态。从 i 到 $N(s)$ 和 $N(t)$ 的起始状态各有一个空转换, 从 $N(s)$ 和 $N(t)$ 到接受状态也各有一个空转换。请注意, $N(s)$ 和 $N(t)$ 的接受状态在 $N(r)$ 中不是接受状态。由于从 i 到 f 的任何路径要么只通过 $N(s)$, 要么只通过 $N(t)$, 并且离开 i 或进入 f 的空转换都不会改变路径上的标号, 因此我们可判定 $N(r)$ 识别 $L(s) \mid L(t)$, 也就是 $L(r)$ 相同。也就是说, 图 3.40 中的 NFA 是一个正确的处理并运算符的构造。

Thompson's Construction Algorithm

The inductive rules: the concatenation case

- **st**: $N(s)$ and $N(t)$ are NFA's for subexpressions s and t

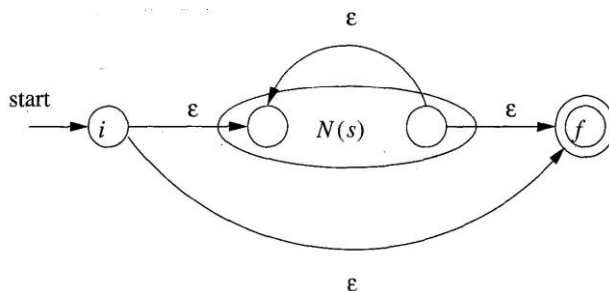


b) 假设 $r = st$ 。然后如图 3.41 所示构造 $N(r)$ 。 $N(s)$ 的起始状态成为 $N(r)$ 的起始状态， $N(t)$ 的接受状态是 $N(r)$ 的唯一接受状态。 $N(s)$ 的接受状态和 $N(t)$ 的开始状态合并为一个状态，合并后的状态拥有原来进入或离开合并前的两个状态的全部转换。图 3.41 中一条从 i 到 f 的路径必须首先经过 $N(s)$ ，因此这条路径的标号以 $L(s)$ 中的某个字符串开始。然后该路径继续通过 $N(t)$ ，因此该路径的标号以 $L(t)$ 中的某个字符串结束。正如我们要论证的，没有转换离开构造得到的接受状态，也没有转换进入开始状态，因此一个路径不可能在离开 $N(s)$ 后重新进入 $N(s)$ 。因此， $N(r)$ 恰好接受 $L(s)L(t)$ ，并且它是 $r = st$ 的一个正确的 NFA。

Thompson's Construction Algorithm

The inductive rules: the Kleene star case

- s^* : $N(s)$ is the NFA for subexpression s

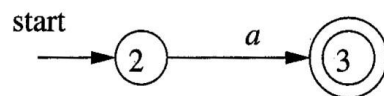


假设 $r = s^*$ 。然后对于 r ，我们构造如图 3.42 所示的 NFA $N(r)$ 。这里， i 和 f 是两个新状态，分别是 $N(r)$ 的起始状态和唯一的接受状态。要从 i 到 f ，我们可以沿着新引入的保号为空的路径前进，这个路径对应于 $L(s)^0$ 中的一个字符串。我们也可以到达 $N(s)$ 的开始状态，然后经过该 NFA，再零次或多次从它的接受状态回到他的开始状态。这些选项允许 $N(r)$ 接受 $L(s)^1$ 、 $L(s)^2$ 等集合中的所有字符串，因此 $N(r)$ 识别的所有串的集合就是 $L(s)^*$ 。

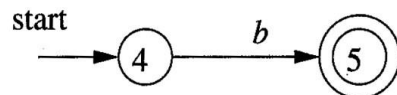
Example

- Use Thompson's algorithm to construct an NFA for the regexp
 $r = (a|b)^*abb$

NFA for the first a (apply basis rule #1)



NFA for the first b (apply basis rule #1)

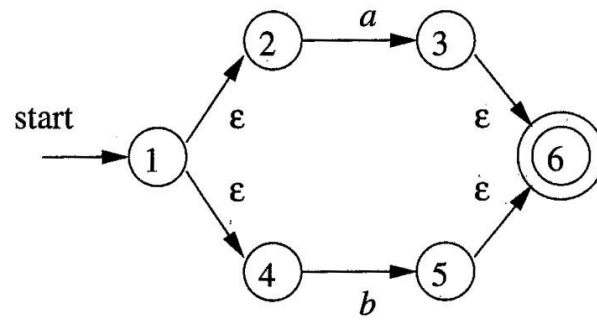


例 3.24: 让我们使用算法 3.23 为 $r = (a|b)^*abb$ 构造一个 NFA。图 3.43 显示了 r 的一颗语法分析树，它类似于第 2.2.3 节中为算术表达式构建的语法分析树。对于子表达式 r_1 ，即第一个 a ，我们构造 NFA:

我们在选择这个 NFA 中的状态编号时考虑了和接下来生成的 NFA 的状态编号之间的一致性。对于 r_2 ，我们构造:

Example Cont.

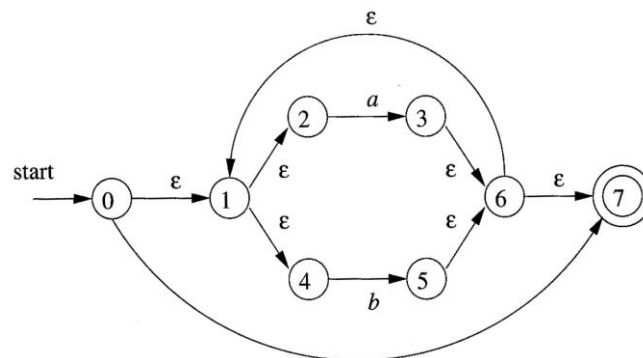
- NFA for $(a|b)$ (apply inductive rule #1)



现在，我们可以使用图 3.40 的构造方法，将 $N(r_1)$ 和 $N(r_2)$ 合并，获得 $r_3 = r_1 | r_2$ 的 NFA；该 NFA 如图 3.44 所示。

Example Cont.

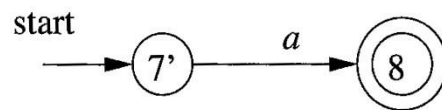
- NFA for $(a|b)^*$ (apply inductive rule #3)



$r4=(r3)$ 的 NFA 与 $r3$ 的 NFA 相同。子表达式 $r5=(r3)^*$ 的 NFA 构造如图 3.45 所示。我们使用图 3.42 所示的方法根据图 3.44 中的 NFA 构造出这个 NFA。

Example Cont.

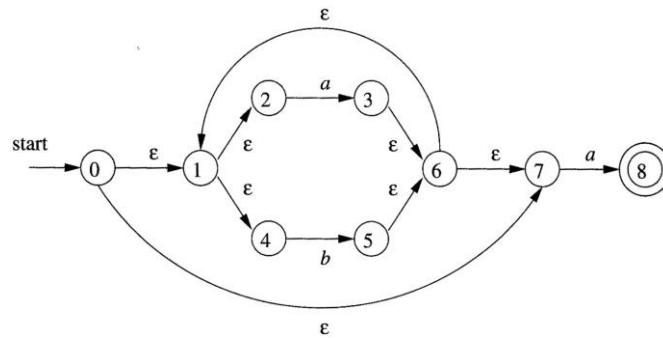
- NFA for the second **a** (apply basic rule #1)



现在，考虑子表达式 $r6$ ，它是表达式中的另一个 a 。我们再次对 a 使用基本构造法，但是我们必须使用新的状态。虽然 $r1$ 和 $r6$ 是相同的表达式，但这个构造方法不允许重复使用我们为 $r1$ 构建的 NFA。 $r6$ 的 NFA 为：

Example Cont.

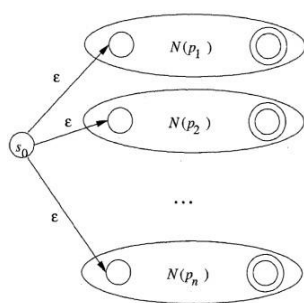
- NFA for $(a|b)^*a$ (apply inductive rule #2) ...



为了获得 $r_7 = r_5 r_6$ 的 NFA，我们应用图 3.41 的构造方法。将状态 7 和 7' 合并，产生图 3.46 的 NFA。继续以这种方式构造出两个分别名为 r_8 和 r_{10} 、对应于子表达式 b 的新 NFA，最终构造出如图 3.34 所示的 $(a|b)^*abb$ 的 NFA。

Combining NFA's

- **Why?** In the lexical analyzer, we need a single automaton to recognize lexemes matching any pattern (in the lex program)
- **How?** Introduce a new start state with ϵ -transitions to each of the start states of the NFA's N_i for pattern p_i



- The language that can be accepted by the big NFA is the union of the languages that can be accepted by the small NFA's

- Different accepting states correspond to different patterns

- 为什么？在词法分析器中，我们需要一个自动机来识别与任何模式（在 lex 程序中）匹配的词目
- 怎么做？为 NFA 的 N 个 i 的每个起始状态引入一个新的起始状态，每个起始状态都有空转换
- 大 NFA 所能接受的语言是小 NFA 所能接受的语言的总和。
- 不同的接受状态对应不同的模式

DFA's for Lexical Analyzers

- Convert the NFA for all the patterns into an equivalent DFA, using the subset construction algorithm
- An accepting state of the DFA corresponds to a subset of the NFA states, in which at least one is an accepting NFA state
 - If there are more than one accepting NFA state, this means that conflicts arise (the prefix of the input string matches multiple patterns)
 - Upon conflicts, find the first pattern whose accepting state is in the set and make that pattern the output of the DFA state

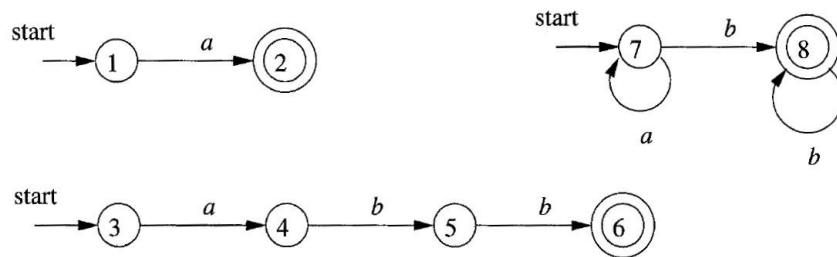
请注意，这三种模式呈现了第 3.5.3 节中讨论过的冲突。更明确地说，字符串 `abb` 同时满足第二个和第三个模式，但是我们将把它看作模式 `p2` 的词素，因为该模式在上面的 Lex 程序中首先列出。然后，输入字符串（例如 `aabbbb`）有许多前缀都满足第三个模式，Lex 规则是取最长的，所以我们继续读取 `b`，直到遇到另一个 `a`，然后我们报告识别的词素就是从第一个 `a` 开始的、包含了其后所有 `b` 的符号串。

词法分析器的 DFA

- 使用子集构建算法，将所有模式的 NFA 转换为等价的 DFA
- DFA 的接受状态对应于 NFA 状态的子集，该子集中至少有一个是 NFA 的接受状态。但是，如果该子集中包含一个以上的 NFA 接受状态，就意味着会出现冲突（输入字符串的前缀匹配多个模式）。此时，解决冲突的方式：按照该接受状态所对应的模式在所有模式中的排序，选择排序最靠前的模式的接受状态作为 DFA 状态的输出

Example

- Suppose we have three patterns: p_1 , p_2 , and p_3
 - a {action A_1 for pattern p_1 }
 - abb {action A_2 for pattern p_2 }
 - a^*b^+ {action A_3 for pattern p_3 }
- We first construct an NFA for each pattern



- 假设我们有三种模式: p , p_2 , 和 p_3
 - a { 行动 A_1 for pattern p }1
 - abb {action A_2 for pattern p }2
 - $A B^+ \{$ 行动 A_3 for pattern p }3
- 我们首先为每个模式构建一个 NFA

Example Cont.

- Combining the three NFA's

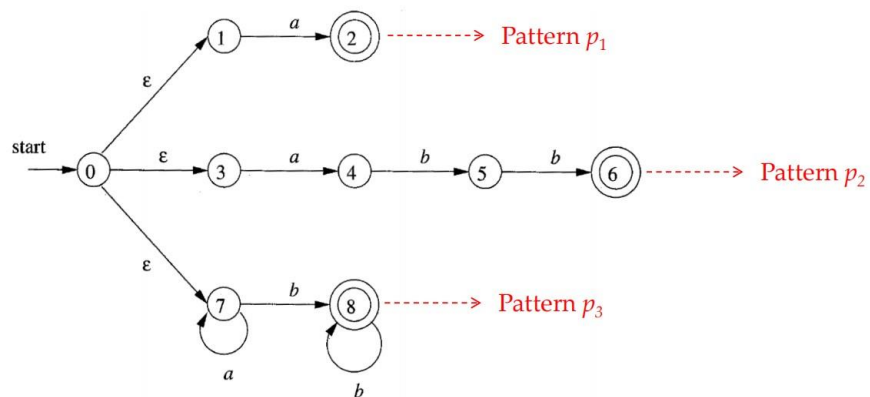


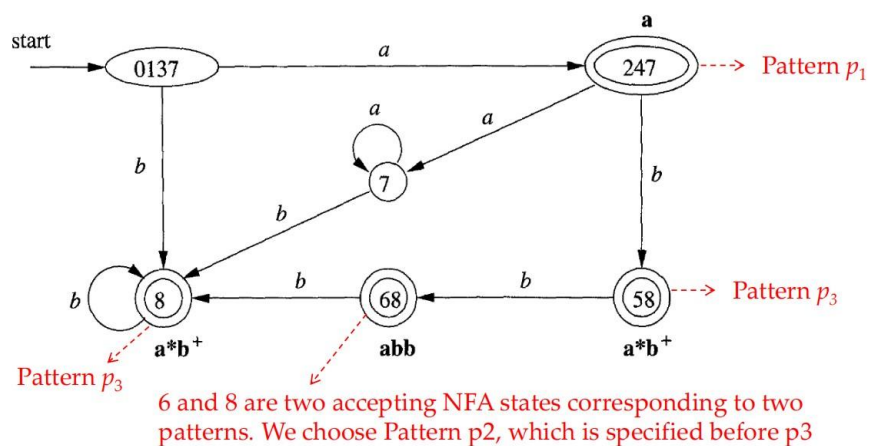
图 3.51 列出了识别这三种模式的三个 NFA。其中第三个是算法 3.23 的转换结果经简化得到的。然后，图 3.52 显示了通过加入一个新开始状态 0 和 3 个空转换将这三个 NFA 合并得到的单个 NFA。

Example Cont.

T	T_a	T_b
$\{0,1,3,7\}$	$\{2,4,7\}$	$\{8\}$
$\{2,4,7\}$	$\{7\}$	$\{5,8\}$
$\{7\}$	$\{7\}$	$\{8\}$
$\{5,8\}$	$\emptyset$	$\{6,8\}$
$\{8\}$	$\emptyset$	$\{8\}$
$\{6,8\}$	$\{1,2,3,6,f\}$	$\{8\}$

Example Cont.

- Converting the big NFA to a DFA



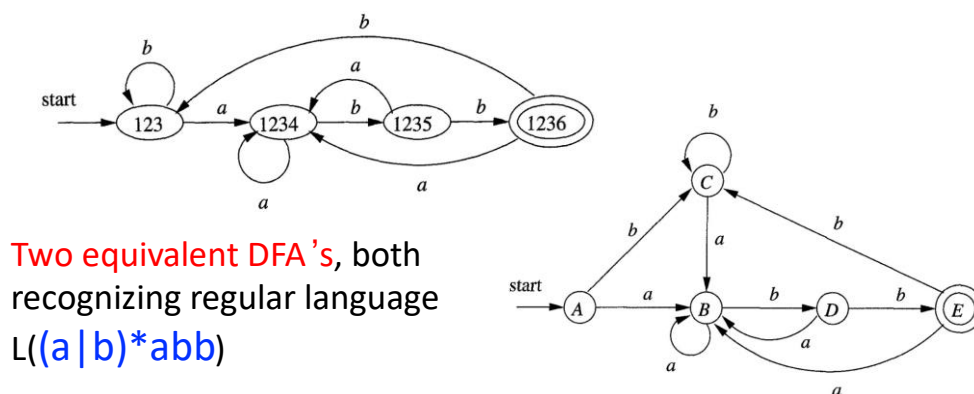
例 3.28: 图 3.54 显示了基于 DFA 的转换图, 该 DFA 是通过图 3.52 中 NFA 的子集构造而构造的。图中的接受状态由该状态所识别的模式作为标号。例如状态 {6; 8} 有两个接受状态, 分别对应于模式 abb 和 a^*b^+ 。由于前者先被列出, 因此该模式就是与状态 {6, 8} 所关联的模式;。

我们在词法分析器中使用 DFA, 就像我们使用 NFA 一样。我们模拟这个 DFA 的运行, 直到某一点上没有后继状态为止 (或者严格来说, 下一个状态是空, 即对应于空的 NFA 状态集合的死状态)。此时, 我们回头查找我们进入过的状态序列, 一旦找到接受状态就执行与该状态相关联的动作。

例 3.29: 假设图 3.54 的 DFA 的输入为 $abba$ 。处理输入时的状态序列为 0137; 247; 58; 68, 并且在读入最后一个 a 时, 没有离开状态 68 的相应转换。因此, 我们从后向前考察这个状态序列, 在这种情况下, 68 本身是一个接受状态, 对应于模式 $p_2 = abb$ 。

Minimizing # States of a DFA

- There can be many DFA's recognizing the same language



可以有多个识别相同语言的 DFA。例如, 请注意图 1 和 2 的 DFA。3.36 和 3.63 都识别语言 $L((a|b)^*abb)$ 。这两个自动机不但各个状态的名字不同, 就连它们的的状态

个数也不一样。如果我们使用 DFA 来实现词法分析器，我们总是希望使用的 DFA 状态数量尽可能少，因为描述词法分析器的转换表需要为每个状态分配条目。

状态名字的问题是次要的。我们可以说，如果只需改变状态名字就可以将一个自动机转换成为另一个自动机，我们就说这两个自动机是同构的。图 3.36 和 3.63 中的两个自动机不是同构的。然而，这两个自动机之间却有着密切的关系。

Minimizing # States of a DFA

- There can be many DFA's recognizing the same language

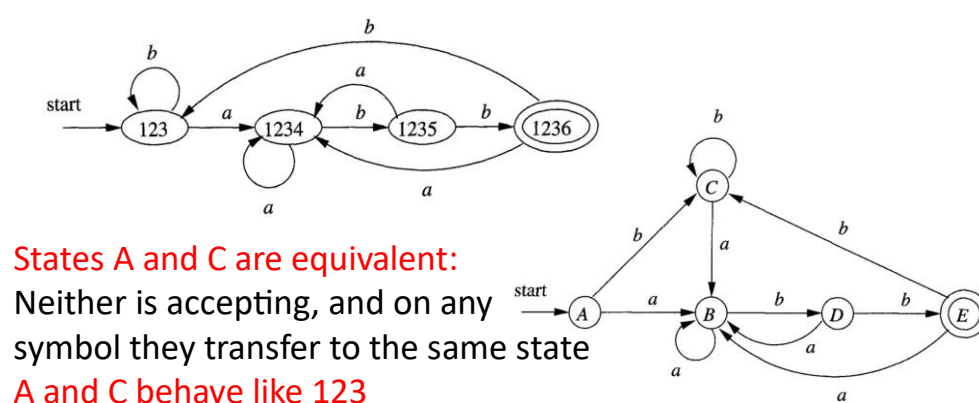


图 3.36 的状态 A 和 C 实际上是等价的，因为它们都不是接受状态，并且对任意输入，它们都会转移到相同的状态——在输入 a 上转到 B，在输入 b 上转到 C。此外，状态 A 和 C 的行为都会图 3.63 中的状态 123 相似。同样，图 3.36 中的状态 B 的行为和图 3.63 中的状态 1234 的行为类似，状态 D 的行为和状态 1235 的行为相似，状态 E 的行为和状态 1236 的行为相似。

Minimizing # States of a DFA Cont.

- There is always a unique minimum-state DFA for any regular language (state name does not matter)
- The minimum-state DFA can be constructed from any DFA for the same language by grouping sets of equivalent states

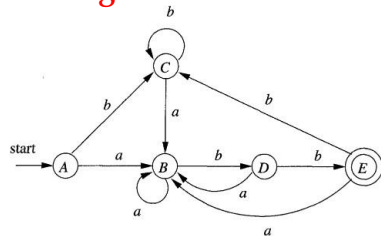
可以得出一个重要的结论，任何正则语言都有一个唯一的（不计同构）状态数目最少的 DFA。此外，从任意一个接受相同语言的 DFA 出发，通过分组合并等价状态，我们总是可以构建得到这个状态数最少的 DFA。

Distinguishing States

- **Distinguishable states**

- We say that string x distinguishes state s from state t if **exactly one** of the states reached from s and t by following the path with label x is an accepting state
- States s and t are **distinguishable** if there exists some string that distinguishes them

- **Indistinguishable states** are equivalent and can be merged



- The empty string ϵ distinguishes any accepting state from any nonaccepting state
- The string bb distinguishes state A from B , since bb takes A to a nonaccepting state C , but takes B to accepting state E

我们将给出一个将任意 DFA 转换为其最小状态等效 DFA 的算法。该算法首先创建输入 DFA 的状态集合的划分。为理解这个算法，我们要了解输入串是如何区分各个状态的。如果分别从状态 s 和 t 出发，沿着标号为 x 的路径到达的两个状态中只有一个是接受状态，则字符串 x 区分状态 s 和状态 t 。如果存在某个能够区分状态 s 和状态 t 的串，那么它们就是可区分的。

- 状态 s 和 t 可区分：

- 存在一个串 x ，使得从 s 和 t 出发的所经历的状态路径中，恰好有一个状态是接受状态，我们就说字符串 x 将状态 s 与状态 t 区分开来。

- 如果存在某种字符串可将状态 s 和 t 区分开来，则这两个状态是可区分的

- 不可区分的状态是等价的，可以合并

空字符串去跟任何接受状态和非接收状态

字符串 bb 将状态 A 与状态 B 区分开来，因为 bb 将 A 带入非接受状态 C ，而将 B 带入接受状态 E 。

DFA State-Minimization Algorithm

Works by partitioning the states of a DFA into groups of states that cannot be distinguished (an iterative process)

- The algorithm maintains a partition (划分), whose groups are sets of states that have not yet been distinguished
- Any two states from different groups are known to be distinguishable
- When the partition cannot be refined further by breaking any group into smaller groups, we have the minimum n -state DFA

状态最小化算法的工作原理是将一个 DFA 的状态集合划分为多个组，每个组中的各个状态之间相互不可区分。然后，将每个组中的状态合并为状态最少 DFA 的一个状态。算法在执行过程中了状态集合的一个划分，分划中的每个组内的各个状态尚不能区分，但是来自不同组的任意两个状态是可区分的。当任意一个组都不能再被分解为更小的组时，这个分划就不能进一步精化，此时我们就有了最小状态 DFA。

最初，该分区由两个组组成：接受状态和不接受状态。算法的基本步骤是从当前分区中取一个状态组，例如 $A = \{s_1; s_2; \dots; s_k\}$ ，并选定某个输入符号 a ，查看 a 是否可以用于区分 A 中的某些状态。我们检查输入 a 上每个 $s_1; s_2; \dots; s_k$ 的转换，如果这些转换达到的状态落入当前分区的两个或多个组，我们就将 A 分成多个组，使得 s_i 和 s_j 在同一组中当且仅当它们在输入 a 上的转换都到达同一个组的状态。我们重复这个分割过程，直到无法根据某个输入符号对任意个组进行分割为止。该思想体现在下面的算法中。

The Partitioning Part

1. Start with an initial partition Π with two groups, F and $S-F$, the accepting and nonaccepting states of D

2. Apply the procedure below to construct a new partition Π_{new}

```
initially, let  $\Pi_{new} = \Pi$ ;  
for ( each group  $G$  of  $\Pi$  ) {  
    partition  $G$  into subgroups such that two states  $s$  and  $t$   
        are in the same subgroup if and only if for all  
        input symbols  $a$ , states  $s$  and  $t$  have transitions on  $a$   
        to states in the same group of  $\Pi$ ;  
    /* at worst, a state will be in a subgroup by itself */  
    replace  $G$  in  $\Pi_{new}$  by the set of all subgroups formed;  
}
```

3. If $\Pi_{new} == \Pi$, let $\Pi_{final} = \Pi$ and the partitioning finishes; Otherwise, $\Pi = \Pi_{new}$ and repeat step 2

1. 首先构造包含两个组 F 和 $S-F$ 的初始划分，这两个组分别是 D 的接受状态组和不接受状态组。

2. 应用图 3.64 的过程构建一个新分区 new 。

最初，让 $new = \Pi$ ； for (的每个组 G) 将 G 划分为子组，使得两个状态 s 和 t 位于同一子组中当且仅当对于所有输入符号 a ，状态 s 和 t 在 a 上有到同一组中状态的转换；/* 在最坏的情况下，状态本身将成为一个子群 */ 将 new 中的 G 替换为形成的所有子群的集合； g 图 3.64：构造 new

3. 如果 $new = \Pi$ ，令 $\Pi_{final} = \Pi$ 并继续步骤 (4)。否则，用 new 替换 Π 并重复步骤 (2)。

The Construction Part

Choose one state in each group of Π_{final} as the **representative** for that group. The representatives will be the states of the minimum state DFA D'

- The start state of D' is the representative of the group containing the start state of D
- The accepting states of D' are the representatives of those groups that contain an accepting state of D
- Establish transitions:
 - Let s be the representative of group G in Π_{final} ; Let the transition of D from s on input a be to state t ; Let r be the representative of t 's group H
 - Then in D' , there is a transition from s to r on input a

4. 在 Π_{final} 的每组中选择一个状态做为该组的代表。这些代表构成了状态最少 DFA D' 的状态。 D' 的其他部分按照如下步骤构建：

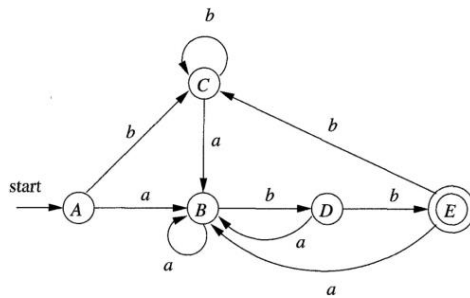
D' 的起始状态是包含 D 的起始状态的组的代表。

D' 的接受状态是那些包含 D 的接受状态的组的代表。请注意，每个组要么只包含接受状态，要么只包含不接受状态，因为我们一开始就将这两类状态分开了，而图 3.64 中的过程总是通过分解已经构造得到的组来得到新的组。

设 s 为 Π_{final} 的某个组 G 的代表，并设 DFA D 从输入 a 上离开状态 s 的转换到达状态 t 。令 r 为 t 所在组 H 的代表。然后在 D' 中存在一个从 s 到 r 在输入 a 上的转换。请注意，在 D 中，组 G 中的每个状态都必须在输入 a 上进入组 H 的某个状态，否则，组 G 将根据图 3.64 的过程分割成更小的组。

Example

- Initial partition: {A, B, C, D}, {E}
- Handling group {A, B, C, D}: b splits it to two subgroups {A, B, C} and {D}
- Handling group {A, B, C}: b splits it to two subgroups {A, C} and {B}
- Picking A, B, D, E as representatives to construct the minimum -state DFA



STATE	a	b
A	B	A
B	B	D
D	B	E
E	B	A

- 初始分区: {a、b、c、d}, {e}
- 处理组 {A、B、C、D}: b 将其分成两个子组 {A、B、C} 和 {D} ——A,B,C 遇到 b 分别转换到 C,D,C,还在{A, B, C, D}这个集合中, 但 D 遇到转换到 E, E 不在{A, B, C, D}中。
- 处理组 {A、B、C}: b 将其分成两个子组 {A、C} 和 {B}—— A,C 遇到 b 分别转换到 C,C,还在{A, B, C}这个集合中, 但 B 遇到 b 转换到 D, D 不在{A, B, C}中。
- 选取 A、B、D、E 作为代表, 构建最小状态 DFA

State Minimization in Lexical Analyzers

The basic idea is the same as the state-minimization algorithm for DFA.

- Differences are:
 - Each accepting state in the lexical analyzer's DFA corresponds to a different pattern. These states are not equivalent.
 - So the initial partition should be: one group of non-accepting states + groups of accepting states for different patterns

词法分析器中的状态最小化

其基本思想与 DFA 的状态最小化算法相同。

- 不同之处在于

词法分析器 DFA 中的每个接受状态都对应一个

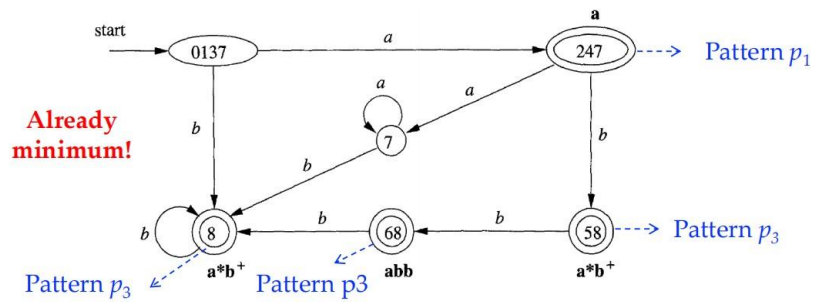
不同的模式。这些状态并不等同。

因此，初始分区应为：一组不接受状态 + 不同模式的接受状态组

Example

Initial partition: $\{0137, 7\}, \{247\}, \{68\}, \{8, 58\}, \{\emptyset\}$

- We add a dead state $\emptyset$: we suppose has transitions to itself on inputs a and b. It is also the target of missing transitions on a from states 8, 58, and 68.



初始分区: $\{0137, 7\}, \{247\}, \{68\}, \{8, 58\}, \{\emptyset\}$

我们添加一个死态 $\emptyset$: 我们假设它在输入 a 和 b 上有向自身的转换。所有输入符号上都转向自己的非接受状态。在一个 DFA 中, 每个状态出发每个输入符号上都会有一个转换, 发现正确词素的时候 DFA 结束。